

Catatan UAS

IF3170 Artificial Intelligence

Semester I 2025/2026

6 Juli 2025

IF '23

Razi Rachman Widyadhana - @zirachw

Muhammad Fathur Rizky - @fathurwithyou

Naufarrel Zhafif Abhista - @nau.farrel_

Apabila terdapat kesalahan pada file, silakan kontak penulis :)

Dilarang membuka dan membawa berkas ini saat ujian berlangsung

This page intentionally left blank

Daftar Isi

0	Preliminary	5
0.1	Dosen Pengampu	6
0.2	Workload	6
0.3	Important Dates	6
0.4	Aturan Perkuliahan	6
0.5	Resources	6
1	Introduction to AI	7
1.1	AI in a Nutshell	8
1.2	4 Approaches in AI Definition	8
1.3	Acting Humanly	9
1.4	Thinking Humanly	9
1.5	Acting Rationally	10
1.6	Thinking Rationally	10
1.7	Latihan	11
2	Agents in AI	12
2.1	What is an Agent?	13
2.2	Agent Model (Studi Kasus)	14
2.3	Rational Agent	15
2.4	PEAS	16
2.5	Task Environment	17
2.6	Agent Structure & Types	18
2.7	Agent Level (Studi Kasus)	21
3	Supervised Learning	23
3.1	What is Supervised Learning?	24
3.2	Classification vs Regression	25
3.3	Overfitting vs. Generalization	25
3.4	Supervised Learning Pipeline	26
4	Decision Tree Learning	27
4.1	Variable (Attribute) Types	28
4.2	What, Why, When DTL?	28
4.3	Algoritma Dasar DTL (ID3)	29
4.4	Issues with DTL	33
5	Linear Models	38
5.1	Linear Regression	39
5.2	Logistic Regression	40
5.3	Proses Training Logistic Regression	41
6	Support Vector Machine	42

6.1	What is SVM	43
6.2	<i>Linear Separability</i> dan <i>Linear Classifier</i>	43
6.3	SVM Objective	44
6.4	SVM for Linearly Separable Data	45
6.5	SVM for Non-Linearly Separable Data	46
6.6	Multi-Class Classification dengan SVM	48
7	Bayes Net	51
7.1	Introduction to Probabilistic Reasoning System	52
7.2	Independence dalam Bayesian Network	53
7.3	Inference dalam Bayesian Network	55
7.4	Learning Bayesian Network dari Data	58
8	Unsupervised Learning (Clustering)	59
8.1	Reviu Unsupervised Learning	60
8.2	Apa itu Clustering?	60
8.3	Tahapan Proses Clustering	62
8.4	Representasi Cluster	62
8.5	Taksonomi Metode Clustering	63
8.6	Partitioning Method: K-Means Clustering	64
	8.6.1 Penentuan Nilai K (The Elbow Method)	67
	8.6.2 Analisis Kelemahan & Solusi	68
8.7	Density-Based Clustering: DBSCAN	68
8.8	Evaluasi Clustering	71
8.9	Internal Measures (Evaluasi Internal)	72
8.10	Studi Kasus Evaluasi Internal	72
	8.10.1 External Measures (Evaluasi Eksternal)	73
	8.10.2 Studi Kasus Evaluasi Eksternal	74
	8.10.3 Perbandingan Kapan Menggunakan Apa	74

Bab 0

Preliminary

0.1	Dosen Pengampu	6
0.2	Workload	6
0.3	Important Dates	6
0.4	Aturan Perkuliahan	6
0.5	Resources	6

Mata kuliah “Inteligensi Buatan” merupakan salah satu mata kuliah wajib prodi di Program Studi Teknik Informatika (S1) STEI ITB.

Mata kuliah ini berada dalam lingkup lab AI (Artificial Intelligence) dengan KK Informatika.

Memiliki beban 4 SKS yang terbagi menjadi 4 jam tatap muka di kelas, 4 jam mengerjakan berbagai tugas, dan 4 jam belajar mandiri. Selain itu, terdapat prerequisite LogKom, Stima, TBFO, dan ProbStat untuk Mata Kuliah ini.

0.1 Dosen Pengampu

- K1: Dr. Nur Ulfa Maulidevi, S.T, M.Sc.
- K2: Dr. Fariska Zakhralativa Ruskanda, S.T., M.T.
- K3: Dr. Eng. Ayu Purwarianti, S.T, M.T.

0.2 Workload

- Latihan Edunex
- 3 Praktikum
- 2 Tugas kecil, 2 Tugas Besar
- Belajar Mandiri (2 Kuis, UTS, UAS)

0.3 Important Dates

- Kuis 1 (Minggu ke-4): 22 September 2025
- UTS (Minggu ke-8)
- Prak 1 (Minggu ke-9): 29 Oktober 2025
- Kuis 2 (Minggu ke-12): 10 November 2025
- Prak 1 (Minggu ke-9): 24 November 2025
- UAS (Minggu ke-16)

0.4 Aturan Perkuliahan

- Syarat lulus IF3170: nilai setiap komponen > 0
- Segala bentuk kecurangan akademik akan mengakibatkan nilai E, bekerja sama antar kelompok berbeda, mengutip dari sumber lain tapi tidak disertakan acuannya (Google, ChatGPT, dll).
- Susulan Kuis, UTS, UAS hanya untuk yang dirawat di rumah sakit/terkena musibah besar.

0.5 Resources

Stuart J Russell & Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Prentice-Hall International, Inc, 2022, Textbook

Bab 1

Introduction to AI

1.1	AI in a Nutshell	8
1.2	4 Approaches in AI Definition	8
1.3	Acting Humanly	9
1.4	Thinking Humanly	9
1.5	Acting Rationally	10
1.6	Thinking Rationally	10
1.7	Latihan	11

1.1 AI in a Nutshell

Is this an AI?



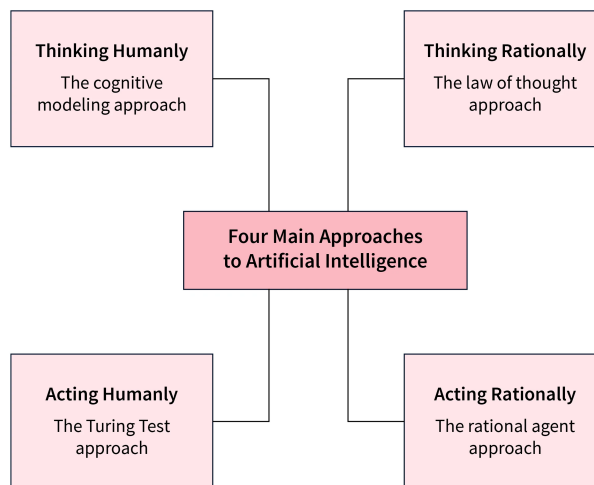
Gambar 1.1: Calculator, obviously

Try to answer these (yes/no) - Quiz IF3170 2024/2025:

1. Lampu yang menyala otomatis ketika ada orang di ruangan yang keberadaannya ditangkap oleh sensor pada lampu tsb
2. Aplikasi penjadwalan kuliah yang menggunakan algoritma pencarian
3. Robot pengantar makanan ke ruangan pasien yang dikendalikan dengan remote control
4. Aplikasi akuntansi yang menggunakan beragam fungsi matematika deterministik dan pelaporan otomatis
5. Aplikasi yang memilah barang catat berdasar gambar yang ditangkap kamera

1.2 4 Approaches in AI Definition

By Russel and Norvig



Gambar 1.2: 4 Approach dalam definisi AI

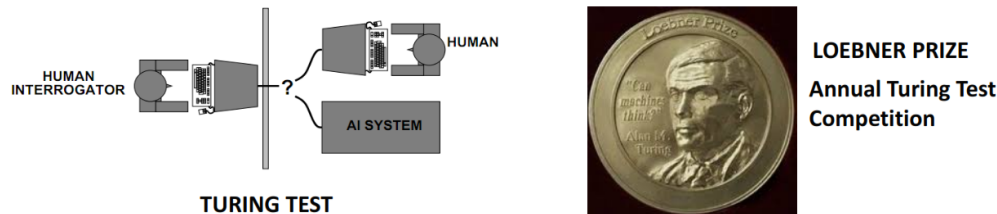
1.3 Acting Humanly

Konsep: The Turing Test Approach

"The art of creating machines that perform functions that require intelligence when performed by people (Kurzweil, 1990)

The study of how to make computers do things at which, at the moment, people are better (Rich and Knight, 1991)

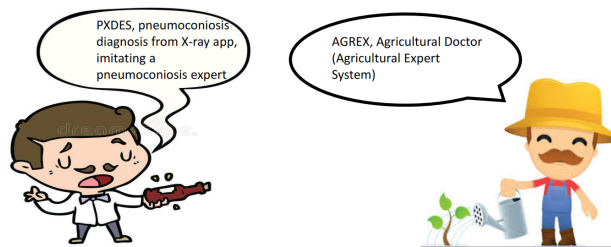
Artinya apa bang messi? Intinya, berfokus pada kemampuan *machines* untuk *mimic* seperti *human* pada umumnya. Bahkan, bisa aja ga bisa bedain ini yang *peragain* apakah *machine* atau *human*.



Gambar 1.3: Turing Test

Turing Test melibatkan seorang juri manusia yang berinteraksi dalam percakapan bahasa alami dengan sebuah mesin dan seorang manusia, **tanpa mengetahui mana di antara keduanya**. Jika mesin berhasil meyakinkan sang juri bahwa ia adalah manusia, maka mesin tersebut dianggap telah lulus Turing Test. Dengan kata lain, merupakan AI dengan pendekatan ***Acting Humanly***

Jadi, contohnya apa? Dari *turing test*, istilah yang sekarang ramai menjadi *chatbot*.



Gambar 1.4: Contoh lain dari AI dengan pendekatan *Acting Humanly* selain *chatbot*

1.4 Thinking Humanly

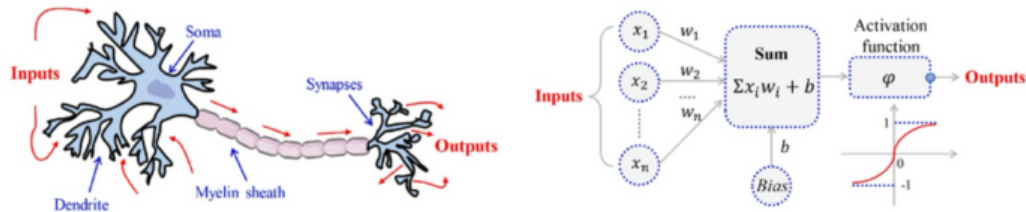
Konsep: The Cognitive Approach

[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ... (Bellman, 1978)

The exciting new effort to make computers think ... machines with minds, in the full and literal sense (Haugeland, 1985)

Pendekatan ini berkaitan dengan proses kognitif internal manusia. Dengan kata lain, bagaimana **manusia berpikir** dan bagaimana kita dapat merepresentasikan proses tersebut **dalam komputer**.

Contohnya? Model *artificial neural network* terinspirasi oleh *neural networks* dalam otak manusia. Meskipun model-model ini tidak sepenuhnya mencerminkan cara kerja otak, mereka berusaha meniru bagaimana *neuron* dalam otak manusia dapat memproses informasi.



Gambar 1.5: (a) Biological Neuron, (b) Artificial Neuron

1.5 Acting Rationally

Konsep

Computational intelligence is the study of the design of intelligent agents (Poole et al., 1998)

AI...is concerned with intelligent behavior in artifacts (Nilsson, 1998)

Berfokus pada hasil akhir dari tindakan AI, yaitu bertindak dengan **cara yang paling optimal atau efisien** untuk mencapai suatu tujuan.

Salah satunya adalah *Pathfinding algorithms* dalam permainan catur. Meskipun mereka mungkin tidak memikirkan permainan dengan cara yang sama seperti manusia, mereka bertujuan untuk membuat langkah terbaik berdasarkan posisi saat ini dan kemungkinan langkah di masa depan.



Gambar 1.6: Contoh AI dengan pendekatan *Acting Rationally*

1.6 Thinking Rationally

Konsep

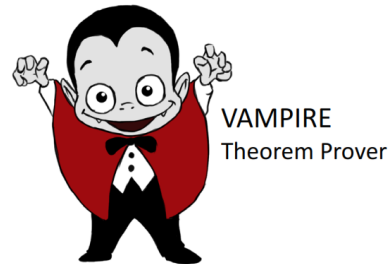
The study of mental faculties through the use of computational models (Charniak and McDermott, 1985)

The study of the computations that make it possible to perceive, reason, and act (Winston, 1992)

Pendekatan ini berkaitan dengan berpikir **secara logis** dan membuat keputusan berdasarkan penalaran. Dengan kata lain, **menggunakan logika** dalam pengambilan keputusan.

Gimana sih bentuknya? seperti **Rule-based systems** yang menggunakan **formal logic** untuk membuat keputusan. Misalnya, expert systems dalam bidang medis yang menganalisis gejala dan data pasien untuk memberikan diagnosis atau rekomendasi perawatan.

Ataupun *theorem solver*, Kenapa?



Gambar 1.7: Contoh AI dengan pendekatan *Acting Rationally*

1.7 Latihan

Berdasarkan empat pendekatan AI, tentukan pendekatan yang digunakan pada aplikasi/teknologi berikut, ataukah aplikasi tersebut tidak menggunakan pendekatan AI.

1. NuPIC, platform perangkat lunak yang berdasarkan pada model struktur dan operasi pada *neocortex* (bagian pada otak manusia)
2. PXDES, aplikasi yang melakukan diagnosis X-ray layaknya seorang pakar melakukan diagnosis, untuk penentuan *pneumoconiosis* (penyakit paru-paru yang disebabkan oleh penghisapan debu)
3. Pc-Nqthm, aplikasi 'proof-checker' yang berlandaskan pada teori automated reasoning, berkaitan dengan aturan formal logika.
4. AceMoney, aplikasi yang membantu mengorganisasikan dan mengatur keuangan individu (mencatat pemasukan dan pengeluaran)
5. Vampire, *automatic theorem prover* untuk *first order logic* yang menjuarai 11 kali *world cup in theorem prover* sejak 1999
6. Robot melakukan eksplorasi pada suatu lingkungan yang belum pernah dikenali sebelumnya. Robot tersebut bekerja untuk sampai pada lokasi tertentu yang diinginkan oleh pemiliknya dari posisi awal mereka diletakkan
7. Logic Problem Solver, aplikasi yang dapat membantu menyelesaikan persoalan logika yang ada di buku atau majalah logic puzzle
8. AGREX, aplikasi sistem pakar yang membantu petani/pembisnis agrikultur dengan memberikan saran yang benar pada saat yang tepat mengenai penjadwalan irigasi, diagnosis penyakit padi, pemupukan, dan perlindungan tanaman
9. Vitamin D, perangkat lunak yang digunakan untuk mendeteksi manusia atau objek bergerak pada video streams. Aplikasi ini memanfaatkan teknologi yang memodelkan *neocortex* (bagian dari otak manusia yang bertanggung jawab untuk high level perception)

Bab 2

Agents in AI

2.1	What is an Agent?	13
2.2	Agent Model (Studi Kasus)	14
2.3	Rational Agent	15
2.4	PEAS	16
2.5	Task Environment	17
2.6	Agent Structure & Types	18
2.7	Agent Level (Studi Kasus)	21

2.1 What is an Agent?

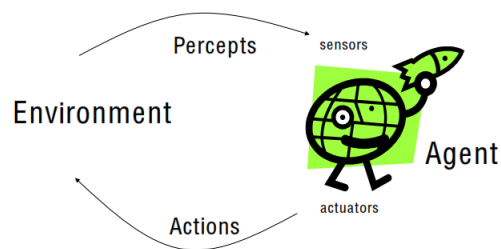
Definisi 2.1

Segala sesuatu yang dapat dipandang sebagai **persepsi** terhadap *environment* melalui *sensors* dan **bertindak** terhadap *environment* tersebut melalui *actuators*.

Dapat berupa:

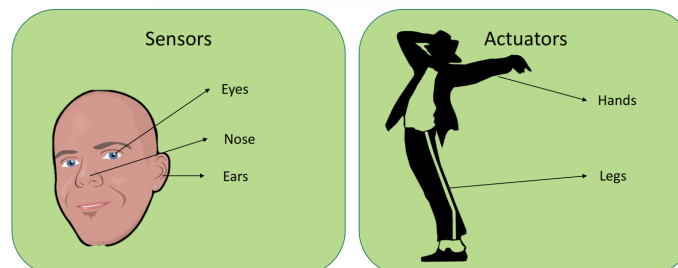
- Robot
- Pabrik
- Program *Web Shopping*

Computational agents yang berperilaku secara otonom.



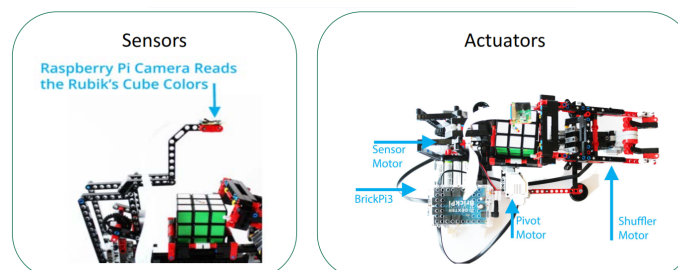
Gambar 2.1: Agent dan Environment

Sebagai contoh pada manusia:



Gambar 2.2: Human Agent

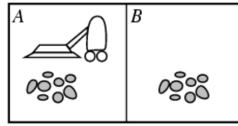
Adapun contoh pada robot:



Gambar 2.3: Rubik Solver Robot Agent

2.2 Agent Model (Studi Kasus)

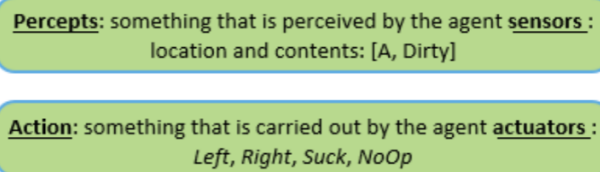
Berikut persoalan yang sederhana, yaitu Agent Penghisap Debu. Terdapat 2 buah ruangan, A dan B. Agent pada awalnya berada pada ruangan A.



Gambar 2.4: Vacuum-Cleaner World

Dari persoalan, maka dapat diperoleh:

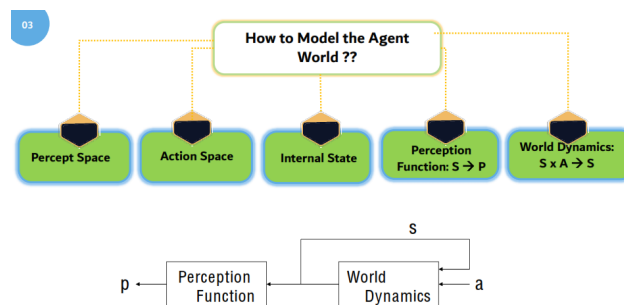
1. Agent dapat menangkap dua hal dari lingkungannya, yaitu lokasi dan keadaan lingkungannya.
2. Aksi yang dapat dilakukan oleh agen adalah bergerak, menghisap, dan *do nothing*



Gambar 2.5: Persepsi dan Aksi Agent

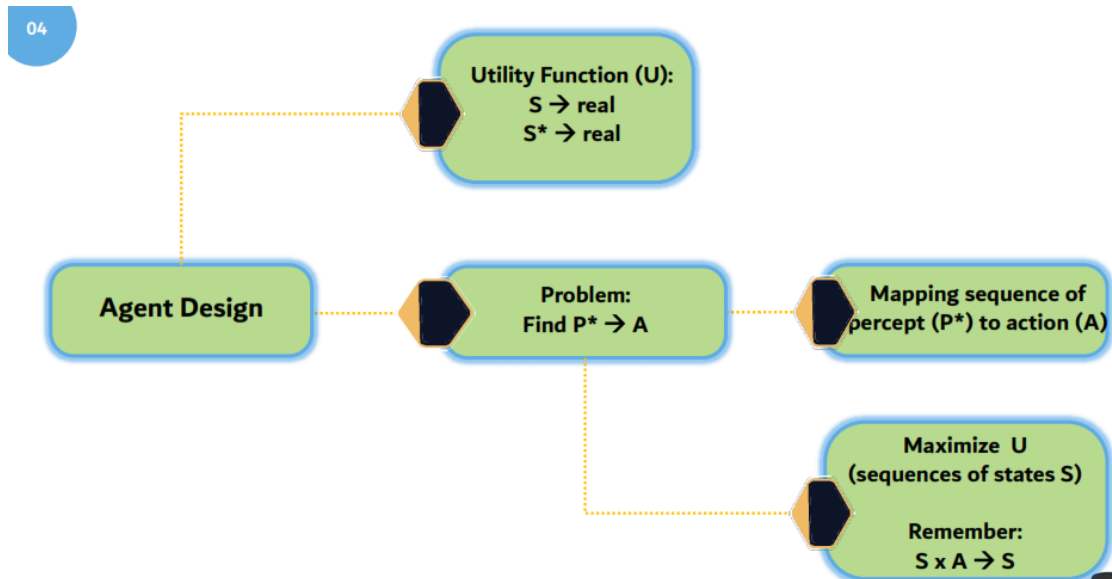
Dari analisis di atas:

1. Agen memiliki kumpulan persepsi atau kondisi atau state yang bisa ditangkap oleh agen yang dirancang ada pada agen (Percept Space)
2. Aksi apa aja yang bisa dilakukan oleh agen untuk mencapai tujuannya juga harus dirancang dari awal (Action Space)
3. Kondisi Internal atau state dalam program atau proses berpikir agen untuk menentukan apakah goal telah tercapai atau belum (Internal Space)
4. Representasi state mungkin berbeda dari kondisi yang ditangkap oleh agen, maka diperlukan fungsi yang memetakan state ke dalam suatu kondisi yang bisa ditangkap oleh agen (Perception Function: $S \rightarrow P$)
5. Dinamika dunia agen, jika pada suatu state lingkungan dilakukan aksi tertentu, maka state lingkungan akan berubah atau tetap (World Dynamics: $S \times A \rightarrow S$)



Gambar 2.6: Pemodelan Dunia Agent

Setelah itu, perlu suatu angka kualitas yang menunjukkan seberapa baik agen yang dirancang dengan menggunakan nilai utilitas.



Gambar 2.7: Evaluasi Agent

2.3 Rational Agent

Agen perlu untuk bekerja dengan baik dengan selalu berpikir untuk memilih aksi terbaik dalam mencapai tujuan yang diinginkan. Ketika agen memilih aksi terbaik, perlu didefinisikan *performance measure*.

Definisi 2.2: Rational Agent

Untuk tiap *sequence percepts* yang mungkin dan modal/bekal pengetahuan yang dimiliki oleh agen, **Rational Agent** harus memilih aksi yang tepat yang dapat memaksimalkan *performance measure*

Notes

Rational tidak berarti maha tahu (omscience).

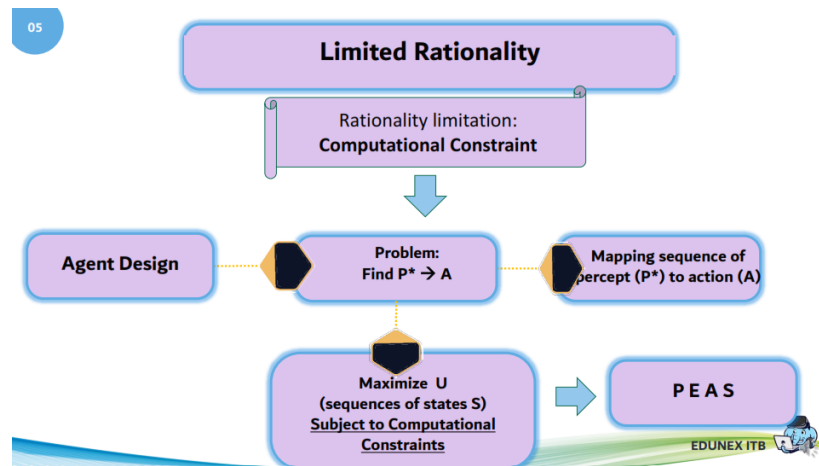
Jika seseorang tidak mendengarkan ramalan cuaca yang menyatakan hari ini akan curah dengan tidak ada hujan. Namun, seseorang tersebut masih membawa payung. Aksi tersebut masih dikatakan rational karena seseorang tidak tahu mengenai ramalan cuaca.

Rational juga tidak sama dengan sukses.

Ketika seseorang tersebut membawa payung agar tidak basah, tetapi seseorang tersebut bertemu dengan anjing di tengah jalan dan menggunakan payung tersebut untuk menghalau anjing.

Seseorang tersebut sukses menggunakan payung untuk menghindari digigit anjing, tetapi tidak rasional karena payung tersebut untuk menghalau hujan, bukan untuk menghalau anjing.

Akibat batasan komputasi, maka definisi desain agen akan berubah pada pemilihan aksi untuk mendapatkan *sequence of states* yang memiliki nilai utilitas maksimal.



Gambar 2.8: Definisi dengan PEAS

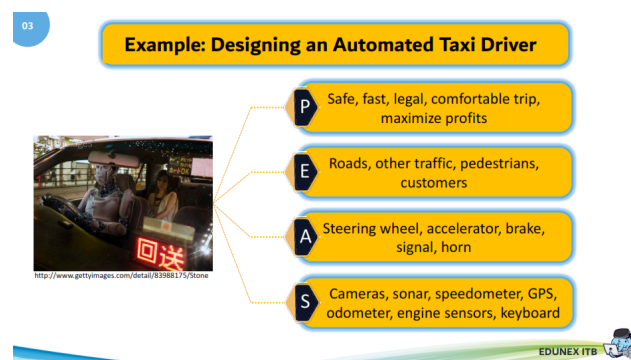
2.4 PEAS

Konsep

PEAS adalah kerangka kerja yang digunakan untuk mendefinisikan lingkungan tugas dalam AI, terdiri dari empat komponen:

1. Performance Measure - Kriteria untuk mengevaluasi keberhasilan agent, seperti akurasi.
2. Environment - Kondisi atau konteks di mana agent beroperasi, seperti dunia nyata atau simulasi.
3. Actuators - Mekanisme yang digunakan agent untuk bertindak, seperti motor atau output teks.
4. Sensors - Alat yang digunakan agent untuk memahami lingkungan, seperti kamera.

PEAS membantu merancang agent dengan memahami kebutuhan dan batasan lingkungan tugas secara terstruktur.

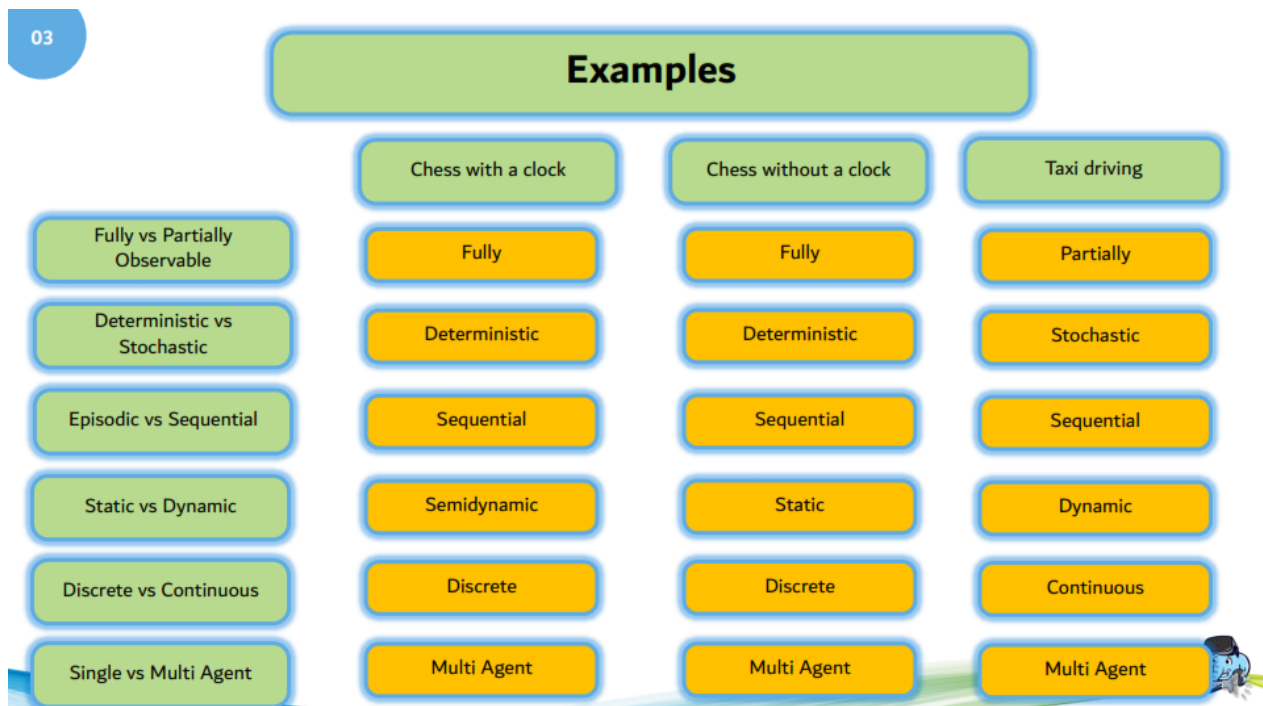


Gambar 2.9: Contoh Abstraksi PEAS

2.5 Task Environment

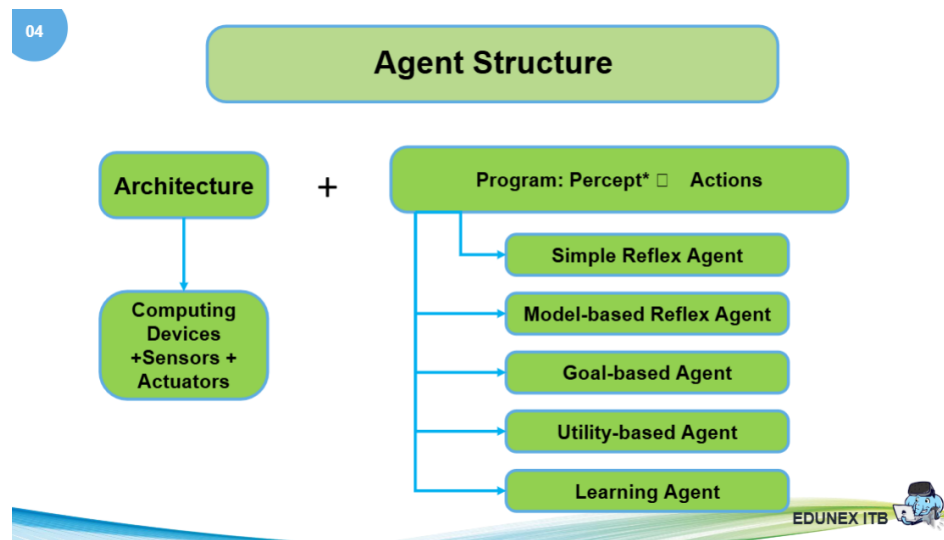
Enam properti lingkungan tugas dalam AI meliputi:

1. Fully Observable vs Partially Observable: Apakah agent memiliki akses penuh ke informasi lingkungan (fully) atau hanya sebagian (partially).
 2. Deterministic vs Stochastic: Apakah hasil tindakan dapat diprediksi (deterministic) atau mengandung elemen acak (stochastic).
 3. Episodic vs Sequential - Apakah tugas terdiri dari episode independen (episodic) atau langkah-langkah yang saling bergantung (sequential).
 4. Static vs Dynamic vs Semidynamic: Apakah lingkungan tetap (static) atau berubah seiring waktu (dynamic) atau lingkungan tetap tetapi performanya dinilai berdasarkan waktu yang digunakan (semidynamic) seperti catur dengan batasan waktu.
 5. Discrete vs Continuous: Apakah state, aksi, dan waktu bersifat diskrit (discrete) atau kontinu (continuous).
 6. Single Agent vs Multi-Agent: Apakah hanya ada satu agent atau beberapa agent yang berinteraksi.
- X. Known vs Unknown: Lebih pada pengetahuan agen terhadap cara kerja lingkungan, keluaran diberikan (known) atau harus dipelajari agen (unknown)



Gambar 2.10: Studi Kasus Task Environment

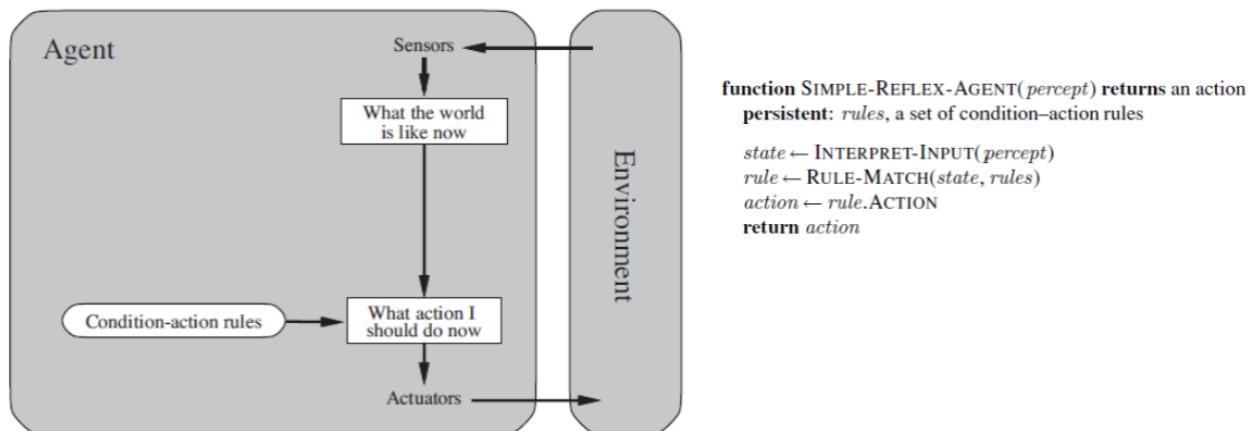
2.6 Agent Structure & Types



Gambar 2.11: Arsitektur dan Tipe-tipe Agent

Konsep: Simple Reflex Agent

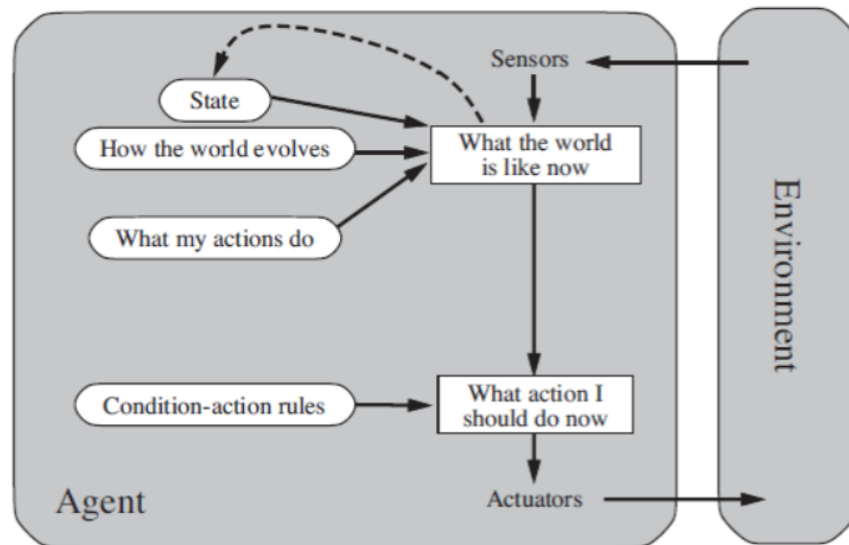
Agent yang bertindak berdasarkan aturan kondisi-aksi sederhana, hanya menggunakan persepsi saat ini (contoh: termostat).



Gambar 2.12: Simple Reflex Agent

Konsep: Model-Based Reflex Agent

Agent yang menggunakan model internal lingkungan untuk melacak state berdasarkan persepsi masa lalu dan saat ini (contoh: vacuum cleaner cerdas).



function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
persistent: *state*, the agent's current conception of the world state
model, a description of how the next state depends on current state and action
rules, a set of condition–action rules
action, the most recent action, initially none

```

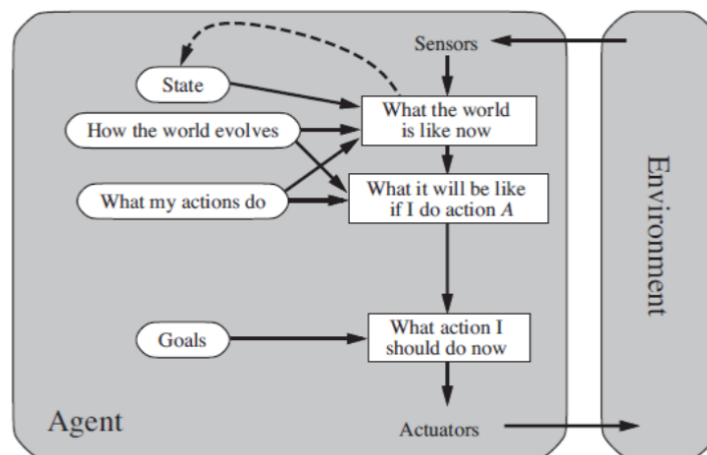
state ← UPDATE-STATE(state, action, percept, model)
rule ← RULE-MATCH(state, rules)
action ← rule.ACTION
return action

```

Gambar 2.13: Model-Based Reflex Agent

Konsep: Goal-Based Agent

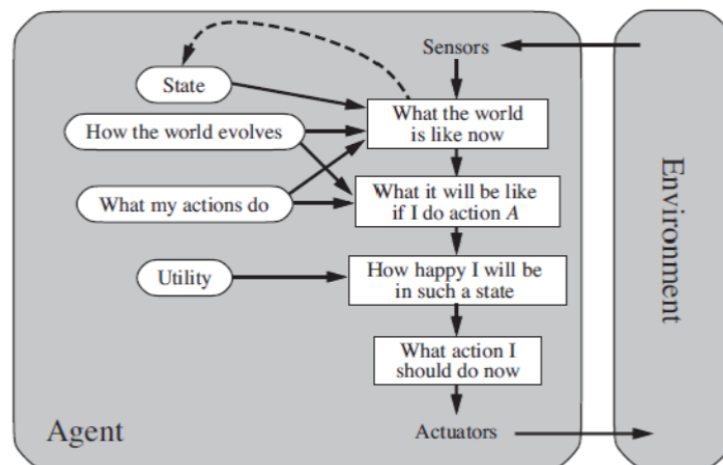
Agent yang bertindak untuk mencapai tujuan tertentu dengan mempertimbangkan konsekuensi aksi (contoh: sistem navigasi GPS).



Gambar 2.14: Goal-Based Agent

Konsep: Utility-Based Agent

Agent yang memaksimalkan utilitas atau kepuasan berdasarkan fungsi utilitas, mempertimbangkan preferensi (contoh: agen e-commerce yang merekomendasikan produk).



Gambar 2.15: Utility-Based Agent

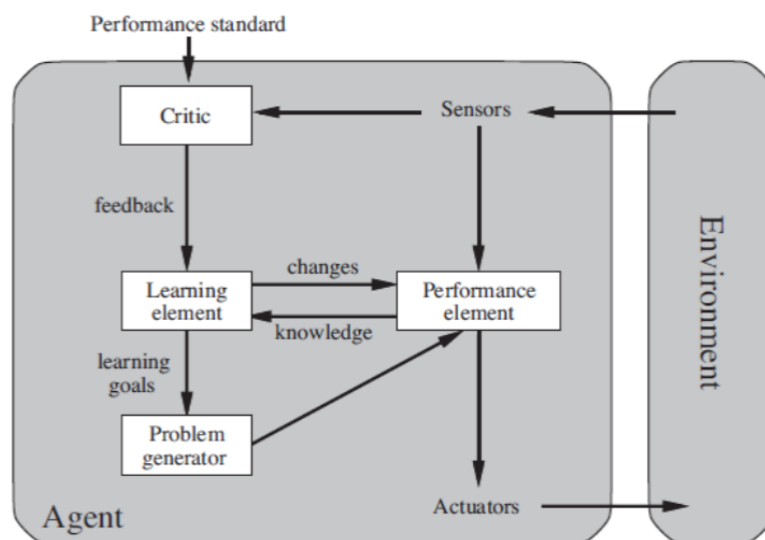
Konsep: Learning Agent

Agent yang dapat belajar dari pengalaman untuk meningkatkan performa seiring waktu, menggunakan komponen seperti kritik dan learning (contoh: agen reinforcement learning untuk permainan).

Critic tells the learning element how well the agent is doing with respect to a fixed performance standard

The **learning element** (responsible for making improvement) uses feedback from the critic on how the agent is doing and determines how the performance element should be modified to do better in the future

Problem Generator is responsible for suggesting actions that will lead to new and informative experiences

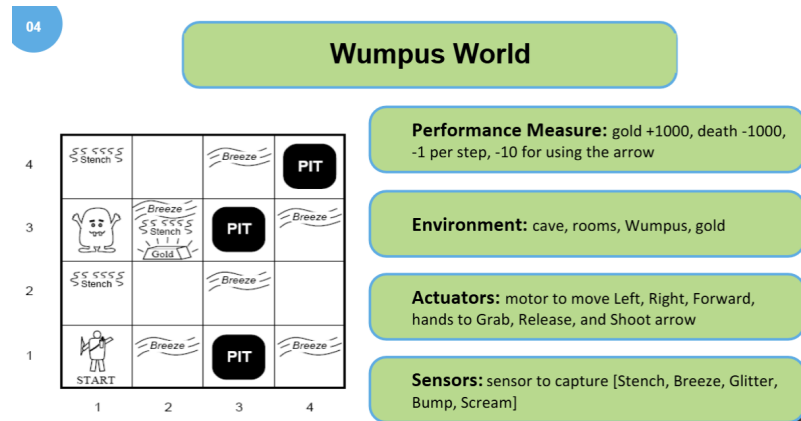


Performance element responsible for element selecting external actions. The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions

Gambar 2.16: Learning Agent

2.7 Agent Level (Studi Kasus)

Untuk memperkenalkan tingkatan pada agent, terdapat persoalan klasik, yaitu Wumpus World. Terdapat goa yang memiliki banyak ruangan, Wumpus akan memakan apa aja dan siapa aja yang masuk ke ruangnya. Wumpus dapat dipanah oleh agen, tetapi agen hanya memiliki satu anak panah. Pada beberapa ruangan goa, terdapat lubang tidak berdasar yang akan menjebak siapa aja kecuali Wumpus. Tujuan dari agent adalah mendapatkan emas yang ada pada salah satu ruangan goa.



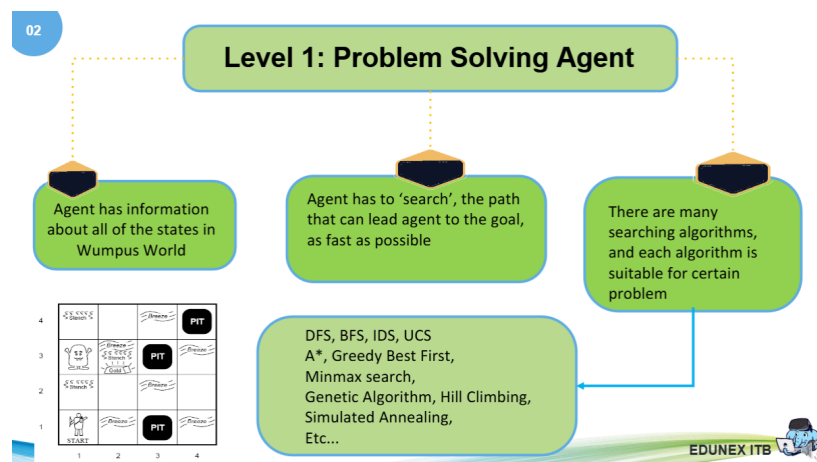
Gambar 2.17: Persoalan Wumpus World dan Abstraksi PEAS-nya

Konsep: Problem Solving Agent (Level 1)

Semua state persoalan yang bisa ditangkap oleh agent dan semua aksi yang bisa dipilih untuk mencapai tujuan telah diberikan.

Tugas agent adalah melakukan pencarian aksi apa yang harus dilakukan agar agent dapat mendapatkan goal dari start.

Agent paling sederhana dalam pemrosesan pencarian solusi dan tidak ada penalaran khusus.

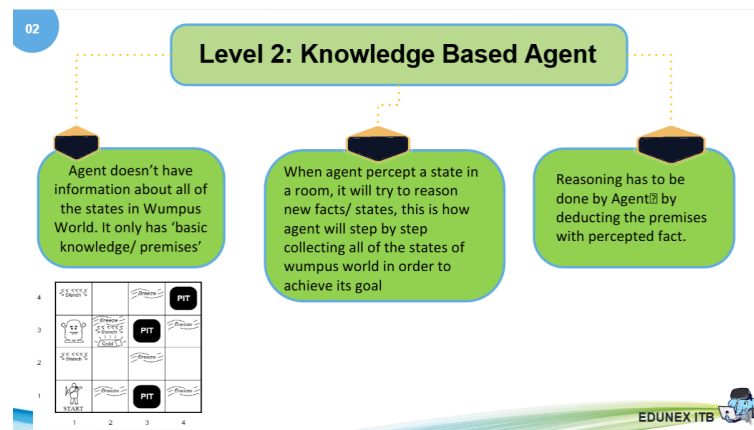


Gambar 2.18: Problem Solving Agent pada persoalan Wumpus World

Konsep: Knowledge Based Agent (Level 2)

Tidak semua state diberikan di awal, namun yang diberikan adalah pengetahuan dasar atau premis yang bisa dimanfaatkan pada proses penalaran untuk mendapatkan fakta atau state yang baru.

Agent telah dilengkapi oleh pengetahuan. Oleh karena itu, agent memiliki kemampuan penalaran untuk memperoleh fakta atau state baru dari dunia persoalan yang dihadapi dalam rangka untuk menyelesaikan persoalan tersebut.

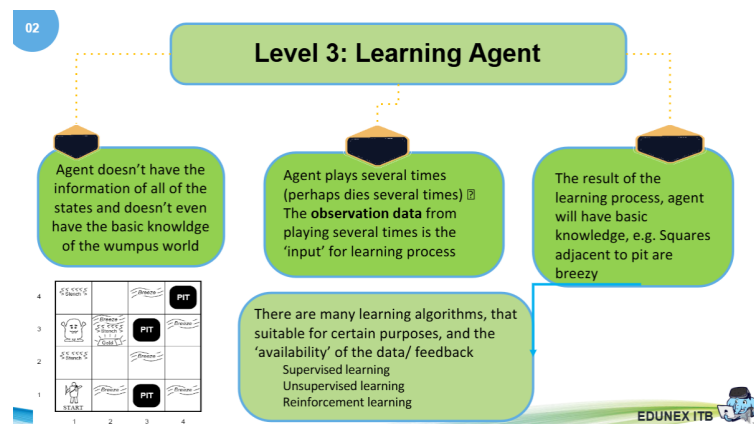


Gambar 2.19: Knowledge Based Agent pada persoalan Wumpus World

Konsep: Learning Agent (Level 3)

Tidak ada informasi yang diberikan kepada agent mengenai states yang ada di persoalan. Modal yang diberikan kepada agent adalah kumpulan data hasil observasi selama beberapa waktu.

Dari kumpulan data tersebut, agent harus dapat membentuk pengetahuan dasar dan menyelesaikan persoalan.



Gambar 2.20: Learning Agent pada persoalan Wumpus World

Bab 3

Supervised Learning

3.1	What is Supervised Learning?	24
3.2	Classification vs Regression	25
3.3	Overfitting vs. Generalization	25
3.4	Supervised Learning Pipeline	26

3.1 What is Supervised Learning?

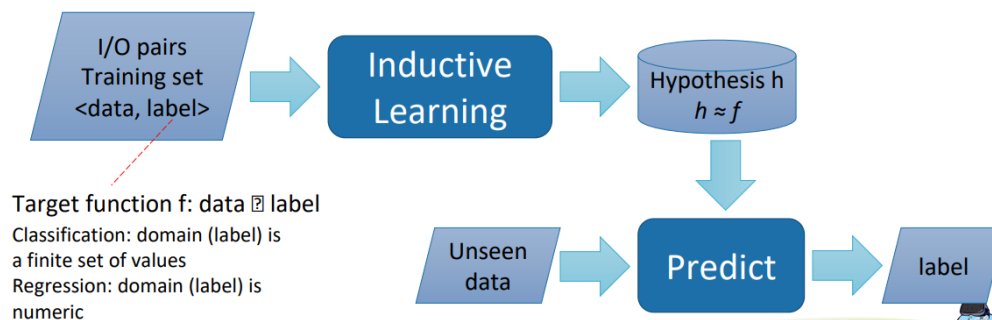
Konsep

Supervised Learning adalah paradigma pembelajaran mesin di mana agen belajar dari sekumpulan data yang sudah diberi **label** dengan benar. Setiap data poin dalam set pelatihan terdiri dari pasangan *input-output*.

Input: Disebut juga fitur, atribut, atau variabel independen (disimbolkan dengan X).

Output: Disebut juga label, target, atau variabel dependen (disimbolkan dengan y atau $f(x)$).

Proses belajar dalam Supervised Learning disebut **Inductive Learning**. Ini adalah proses untuk menyimpulkan sebuah fungsi umum dari contoh-contoh spesifik.



Gambar 3.1: Inductive Learning dalam Supervised Learning

Dalam hal ini:

- Fungsi Target (f): Ini adalah fungsi “ideal” atau “sebenarnya” yang memetakan input ke output. Dalam dunia nyata, kita tidak pernah tahu bentuk pasti dari fungsi ini.
- Hipotesis (h): Ini adalah fungsi yang dipelajari oleh algoritma dari data latih. Tujuannya adalah agar h bisa mengaproksimasi atau mendekati f ($h \approx f$) sebaik mungkin.
- Training Set: koleksi dari pasangan $\langle data, label \rangle$ ini. Analogi sederhananya adalah belajar dengan seorang “guru” yang memberikan soal (*input*) beserta kunci jawabannya (*output*).

Alur prosesnya adalah:

1. Algoritma menerima Training Set.
2. Melalui proses Inductive Learning, algoritma menghasilkan sebuah Hipotesis h .
3. Hipotesis h ini kemudian digunakan untuk membuat prediksi pada data baru yang belum pernah dilihat (*unseen data*).

3.2 Classification vs Regression

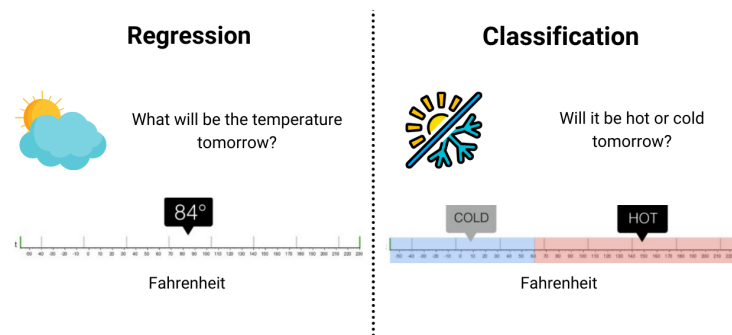
Supervised Learning umumnya dibagi menjadi dua jenis tugas utama, tergantung pada domain dari label/output-nya:

1. Klasifikasi (Classification):

Tujuannya adalah memprediksi label kategori/diskrit. Domain dari labelnya adalah sebuah himpunan nilai yang terbatas. Contoh: Memprediksi apakah sebuah email adalah 'spam' atau 'bukan spam'; mengklasifikasikan spesies bunga ('setosa', 'versicolor', 'virginica').

2. Regresi (Regression):

Tujuannya adalah memprediksi nilai numerik/kontinu. Domain dari labelnya bisa berupa nilai riil apa pun. Contoh: Memprediksi harga rumah berdasarkan luasnya; memprediksi suhu besok hari.



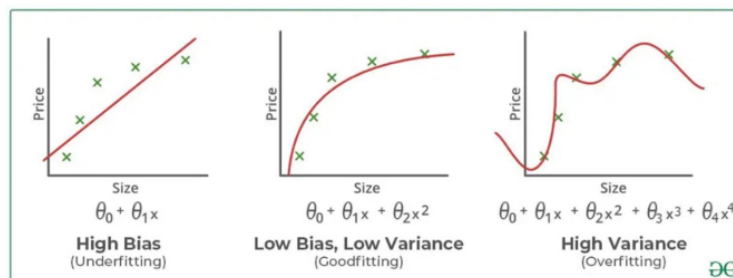
Gambar 3.2: Classification vs Regression

3.3 Overfitting vs. Generalization

Konsep

Model Sederhana (misal: Garis Lurus): Mungkin membuat beberapa kesalahan pada data latih, tetapi cenderung lebih *robust*.

Model Sangat Kompleks: Dapat menciptakan decision boundary yang sangat berliku-liku untuk mengklasifikasikan semua titik di data latih dengan sempurna (akurasi training 100%). Namun, model ini menjadi terlalu “menghafal” data latih, termasuk *noise*-nya. Ini disebut Overfitting.



Gambar 3.3: Bias vs Variance dalam Performa Model

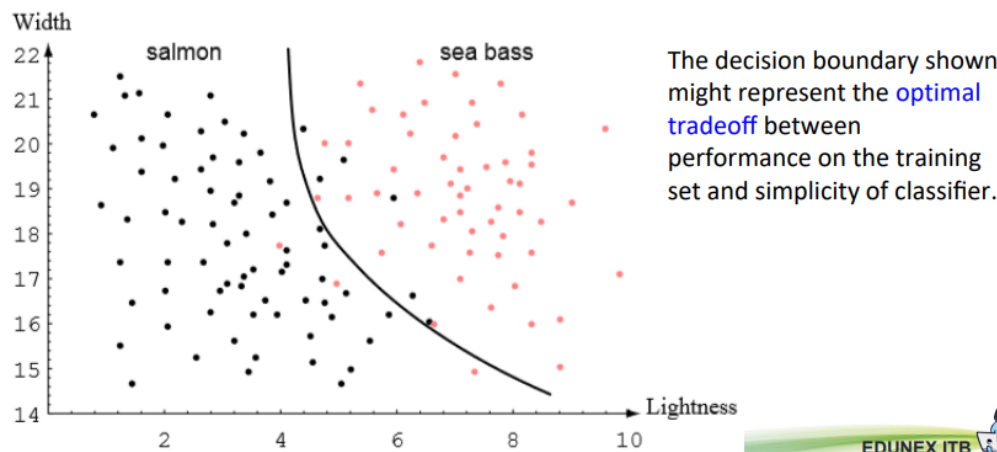
Notes

Konsekuensi *Overfitting*

ketika model ini dihadapkan pada data baru (yang tidak ada dalam set pelatihan), performanya akan sangat buruk. Ia gagal menangkap pola yang mendasari dan hanya menghafal contoh spesifik.

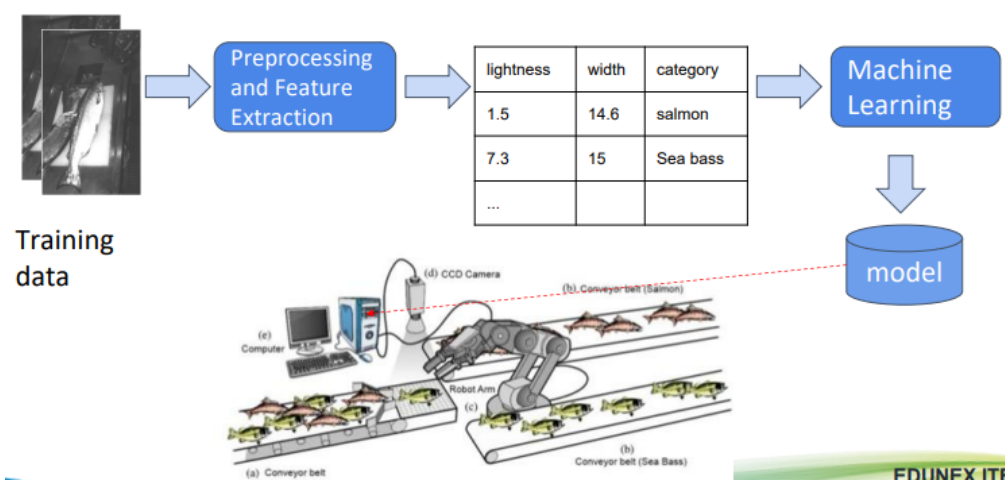
Model Optimal

Model yang baik menemukan keseimbangan. Cukup kompleks untuk menangkap pola yang sebenarnya dalam data, tetapi tidak terlalu kompleks sehingga ia juga menangkap noise. Model seperti ini memiliki kemampuan generalisasi (*generalization*) yang baik, artinya berkinerja baik pada data baru yang belum pernah dilihat.



Gambar 3.4: Workflow Supervised Learning

3.4 Supervised Learning Pipeline



Gambar 3.5: Workflow Supervised Learning

Bab 4

Decision Tree Learning

4.1	Variable (Attribute) Types	28
4.2	What, Why, When DTL?	28
4.3	Algoritma Dasar DTL (ID3)	29
4.4	Issues with DTL	33

4.1 Variable (Attribute) Types

Konsep

Sebelum membangun model, harus dipahami tipe data. Atribut (fitur) data umumnya dibagi menjadi:

1. Numeric (Quantitative): Berupa angka yang bisa diukur.

- Interval-Scaled: Angka yang punya urutan dan unit yang sama (misal: Temperatur, Tanggal). Dapat dihitung selisihnya, tapi “0” tidak berarti “tidak ada”.
- Ratio-Scaled: Seperti interval, tapi punya titik “no” yang absolut (misal: Berat, Umur, Gaji). Dapat disebut “20kg adalah dua kali lipat 10kg”.

2. Categorical (Qualitative): Berupa simbol/kode/status

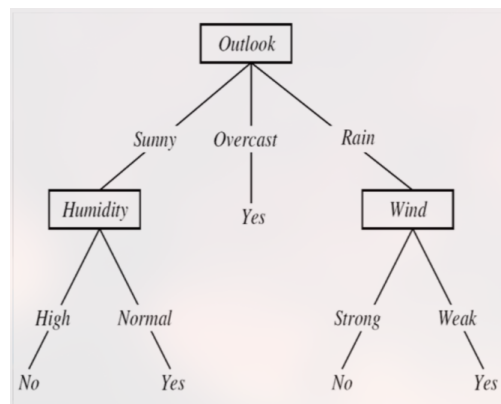
- Nominal: Simbol atau nama (misal: Occupation Type, Zip Code, Student_ID). Angka di sini (spt Zip Code) tidak bisa dioperasikan secara matematis.
- Binary: Hanya punya 2 state (misal: Gender, HIV Positive/Negative). Terdiri lagi atas
Symmetric: Kedua state sama pentingnya (misal: Laki-laki/Perempuan).
Asymmetric: Satu state lebih penting/langka (misal: HIV Positive lebih penting daripada Negative).
- Ordinal: Punya urutan atau ranking yang bermakna, tapi jarak antar nilai tidak diketahui (misal: Drink Size (Small, Medium, Large), Grades (A, B, C), Customer Satisfaction (1-5)).

4.2 What, Why, When DTL?

Definisi 4.1: Decision Tree Learning

DTL adalah salah satu metode Supervised Learning untuk mengaproksimasi (memperkirakan) sebuah fungsi target yang bernilai diskrit.

Model yang dihasilkannya disebut Decision Tree (Pohon Keputusan). Pohon ini juga bisa dengan mudah diubah menjadi satu set aturan IF-THEN.



Gambar 4.1: Visualisasi Pohon Decision Tree Learning

What DTL?

Representasi DTL terdiri dari:

- Internal Node (Simpul Internal): Merepresentasikan sebuah tes pada satu atribut.
- Branch (Cabang): Merepresentasikan salah satu nilai dari atribut tersebut (misal: cabang Sunny).
- Leaf Node (Simpul Daun): Merepresentasikan hasil klasifikasi (misal: Play Tennis = Yes).

Sebuah path dari root ke leaf merepresentasikan sebuah aturan konjungsi (AND). Keseluruhan pohon merepresentasikan disjunction of conjunctions (OR dari aturan-aturan AND).

Why DTL?

- Mudah Dipahami: Hasil pohonnya sangat intuitif dan mudah dijelaskan (interpretability tinggi).
- Direpresentasikan sebagai pohon keputusan ataupun set aturan IF-THEN
- Robust terhadap *Noisy* Data: Cukup tahan terhadap data yang error.
- Mampu Mempelajari Ekspresi *Disjunctive*: Bisa menangani logika (A AND B) OR (C AND D).

When DTL?

DTL sangat cocok untuk masalah di mana:

- Data direpresentasikan sebagai pasangan atribut-nilai. (Untuk atribut kontinu, dilakukan *discretization*)
- Target Function (Kelas) memiliki nilai diskrit (misal: Yes/No, Kucing/Anjing).
- Dibutuhkan deskripsi disjuntif
- Data latih mengandung errors (noise) atau missing values. (Diatasi dengan penggunaan isu DTL)

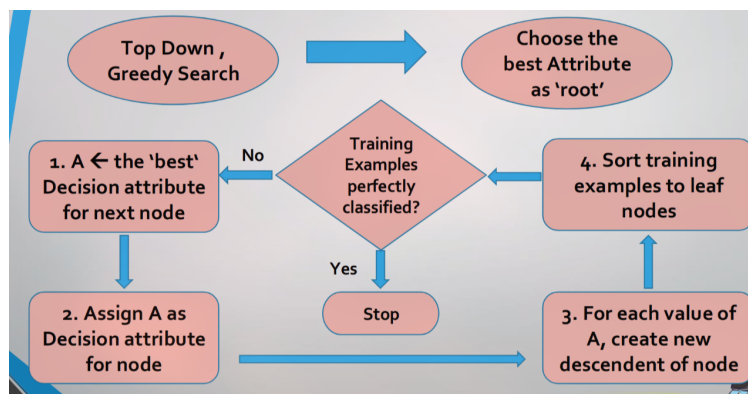
4.3 Algoritma Dasar DTL (ID3)

Definisi 4.2

ID3 (Iterative Dichotomiser 3) adalah algoritma DTL klasik yang membangun pohon menggunakan pendekatan Top-Down dan Greedy Search.

Top-Down: Membangun pohon dari root (atas) ke leaf (bawah).

Greedy: Di setiap langkah, dipilih keputusan terbaik saat itu juga (optimum lokal) tanpa memikirkan apakah keputusan itu akan optimal secara global.



Gambar 4.2: Pseudocode Algoritma ID3

Tahapan ID3 sebagai berikut:

1. Mulai dari root node.
2. Pilih “best attribute” (atribut terbaik) untuk memecah data.
3. Jadikan atribut tersebut sebagai node keputusan.
4. Buat cabang (branch) baru untuk setiap nilai dari atribut tersebut.
5. Bagi (sortir) data latih ke cabang-cabang yang sesuai.
6. Ulangi proses ini (rekursif) untuk setiap cabang (yang sekarang menjadi subset data) sampai *stopping condition* terpenuhi.

Sebuah cabang akan berhenti (*stopping condition* dan menjadi leaf node jika:

1. Semua contoh di cabang itu memiliki kelas yang sama (misal: semua ‘Yes’). Ini disebut pure node.
2. Tidak ada atribut lagi yang bisa digunakan untuk memecah data. Jika ini terjadi, leaf node mengambil kelas mayoritas dari data di cabang itu (disebut PLURALITY-VALUE).
3. Tidak ada contoh data di cabang itu. Jika ini terjadi, leaf node mengambil kelas mayoritas dari parent-nya.

```

function DECISION-TREE-LEARNING(examples, attributes, parent_examples) returns
a tree

if examples is empty then return PLURALITY-VALUE(parent_examples)
else if all examples have the same classification then return the classification
else if attributes is empty then return PLURALITY-VALUE(examples)
else
   $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
  tree  $\leftarrow$  a new decision tree with root test A
  for each value  $v_k$  of A do
    exs  $\leftarrow \{e : e \in \text{examples} \text{ and } e.A = v_k\}$ 
    subtree  $\leftarrow$  DECISION-TREE-LEARNING(exs, attributes - A, examples)
    add a branch to tree with label (A =  $v_k$ ) and subtree subtree
  return tree
  
```

Information gain
Gain ratio

$|A|=2$: binary tree
 $|A| \geq 2$: n-ary tree

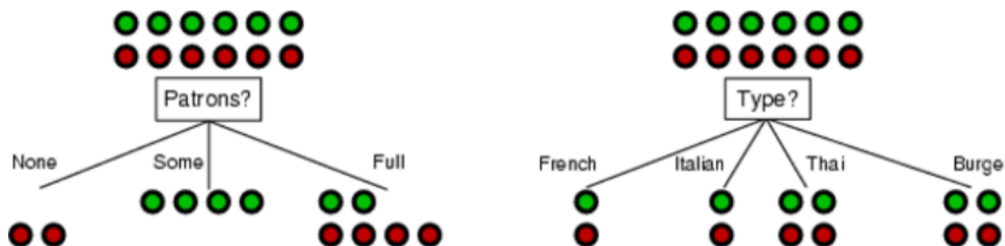
The function PLURALITY-VALUE selects the most common output value among a set of examples, breaking ties randomly.

Gambar 4.3: Pseudocode Algoritma ID3

Konsep: Best Attribute

ID3 memilih atribut yang ideal memisahkan data menjadi subset-subset yang pure (murni satu kelas). Ukuran untuk ini disebut Information Gain.

Untuk menghitung Information Gain, harus dipahami Entropy terlebih dahulu.



Gambar 4.4: *Patrons* merupakan atribut yang lebih baik daripada *Type*

Definisi 4.3: Entropy (Information Content)

Entropy adalah suatu pengukuran dalam Information Theory yang menyatakan nilai ketidakmurnian dari sembarang kumpulan data. Dalam hal ini nanti adalah *training examples*.

Misalkan S adalah himpunan dari *training examples*, entropy dari S menyatakan seberapa banyak bit informasi yang dibutuhkan atau yang harus dikirimkan agar ketika mengambil sebuah data dari S secara acak langsung bisa mengetahui kelas dari data tersebut.

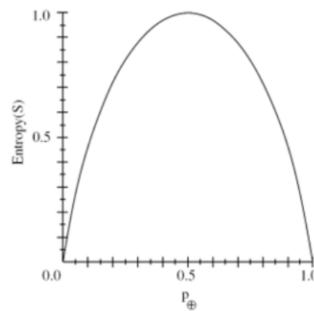
Formulanya adalah total dari negatif proporsi atau bagian S dengan kelas i tertentu dikalikan $\log_2$ dari proporsi tersebut. Proporsi dari tiap kelas nilainya minimal 0 dan maksimal 1.

$$\text{Entropy}(S) = E(S) \equiv \sum_{i=1}^c -p_i \log_2 p_i$$

Dalam kasus 2 kelas:

$$E(S) \equiv -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

S berisi sampel positif dan sampel negatif



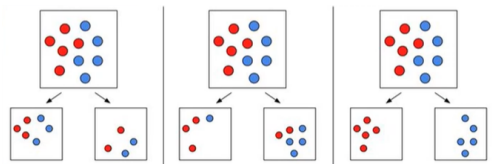
Gambar 4.5: Grafik kemungkinan nilai kelas positif dengan nilai *entropy*-nya

Jika proporsi kelas positif bernilai 0, artinya *training examples* pada S berupa kelas negatif dan proporsi kelas negatif bernilai 1. Ketika dimasukkan pada persamaan *entropy*, maka *entropy* dari S akan bernilai 0.

Dengan kata lain, nilai *impurity* dari dataset S adalah 0. Ketika diambil sembarang data dari S , diketahui kelasnya negatif sehingga tidak perlu dikirimkan informasi apapun untuk bisa menentukan dari data yang diambil tersebut. Hal ini berlaku juga sebaliknya, apabila proporsi kelas positif bernilai 1 maka proporsi kelas negatif 0.

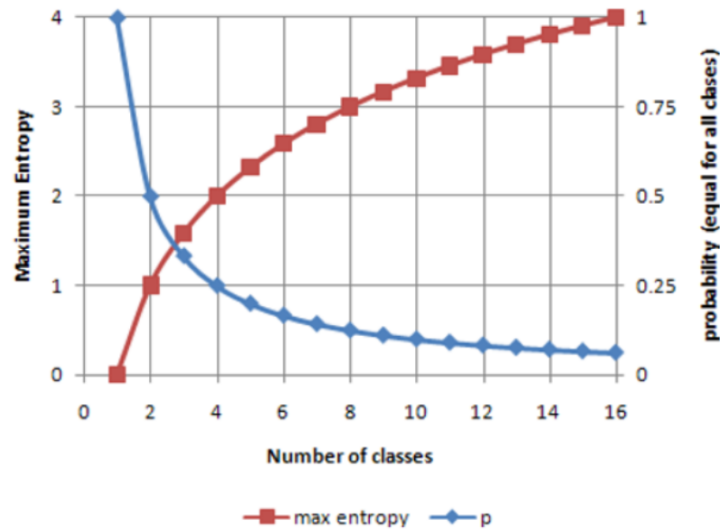
Berbeda ketika proporsi kelas positif bernilai 0.5, yang artinya setengah *training examples* berkelas positif dan setengahnya lagi berkelas negatif. Ketika diambil *instance* secara sembarang dari S , diperlukan informasi sejumlah 1 bit untuk mengetahui kelas dari *instance* yang diambil tersebut.

Kaitannya dengan DTL adalah ingin dipilih *training examples* sedemikian supaya nilai *entropy*-nya sekecil mungkin. Tanpa perlu mengirimkan banyak informasi, bisa diketahui kelas dari suatu *instance*.



Gambar 4.6: Kemungkinan pembagian berdasarkan atribut untuk mendapatkan *entropy* terkecil

Dalam kasus lebih dari 2 kelas:



Gambar 4.7: Entropy terbesar dapat mencapai $\log_2 n$

Lalu, apa kaitannya dengan Information Gain?

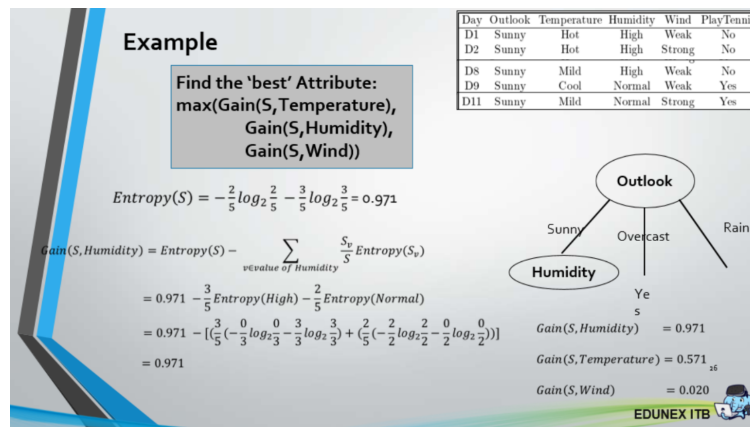
Definisi 4.4: Information Gain

Nilai *gain* digunakan untuk memilih atribut terbaik pada algoritma ID3. Jadi, *gain* dari suatu atribut pada *training examples* S adalah ekspektasi pengurangan nilai *entropy* dataset S jika dikelompokkan datanya berdasarkan nilai pada atribut A .

Akibat diinginkan nilai *entropy*-nya makin kecil, maka dicari atribut dengan *gain*-nya atau ekspektasi pengurangannya terbesar sehingga akan bisa mengurangi nilai *entropy* semula dengan cukup besar.

$$\text{Gain}(S, A) \equiv \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Atribut terbaik adalah atribut dengan nilai *gain* terbesar.



Gambar 4.8: Ilustrasi pencarian *Best Attribute*

4.4 Issues with DTL

Konsep: Overfitting

Secara formal, *Overfitting* terjadi ketika sebuah hipotesis (model) h memiliki error yang lebih rendah pada data latihan daripada hipotesis alternatif h' .

$$\text{error}_{\text{train}}(h) < \text{error}_{\text{train}}(h')$$

Akan tetapi, $h(x)$ memiliki error yang lebih tinggi pada keseluruhan distribusi data (data baru) D .

$$\text{error}_D(h) < \text{error}_D(h')$$

Overfitting dapat disebabkan baik data *noise* maupun data yang dipelajari cukup sedikit.

Sebagai kesimpulan, hipotesis h hanya sesuai untuk *training* data saja.

Yang perlu dilakukan adalah menghasilkan pohon yang seharusnya cukup sederhana dengan akurasi yang cukup baik, terdapat 2 pendekatan:

1. Pre-Pruning (Stop Awal)

- Mekanisme: Menghentikan pertumbuhan tree lebih awal, sebelum ia mencapai titik di mana ia mengklasifikasikan data latihan dengan sempurna.
- Pros: Lebih langsung (cepat) dan *direct*.
- Cons: Sulit untuk memperkirakan secara tepat kapan harus berhenti.

2. Post-Pruning (Pangkas Nanti)

- Mekanisme: Biarkan tree tumbuh hingga overfit, lalu pangkas tree tersebut.
- Pros: Terbukti lebih sukses dalam praktiknya.
- Cons: Membutuhkan lebih banyak langkah dan perlu kriteria untuk menentukan ukuran tree final yang “tepat”.

Pendekatan untuk menentukan Ukuran Tree Final (untuk Post-Pruning):

1. Gunakan Validation Set:

Pisahkan data. Misal, 2/3 untuk *training set* dan 1/3 untuk *validation set*. Gunakan *validation set* untuk mengevaluasi dampak dari pemangkasan.

2. Gunakan Uji Statistik:

Gunakan semua data untuk training, lalu lakukan uji statistik (misal, *chi-square test*) untuk memeriksa apakah memangkas atau mengembangkan sebuah simpul akan menghasilkan peningkatan yang signifikan.

3. Gunakan Minimum Description Length (MDL):

Cari tree yang paling sederhana (kompleksitas rendah) yang masih bisa menjelaskan data dengan baik.

$$h_{\text{MDL}} = \arg \min_{h \in H} (L_{c_1}(h) + L_{c_2}(D | h))$$

Dalam hal ini:

$L_{c_1}(h)$: Panjang (*number of bits*) dari hipotesis *encoding*

$L_{c_2}(D | h)$: Panjang data D dengan hipotesis *encoding* h

Teknik-teknik pada Post-Pruning

1. Reduced Error Pruning

Ini adalah pendekatan yang efektif jika tersedia banyak data.

- Bagi data menjadi training dan validation set.
- Tumbuhkan tree sampai fit dengan data training.
- Lakukan berulang kali (Do until...):
Evaluasi dampak pemangkasan setiap node (menggantinya dengan klasifikasi paling umum di node itu) terhadap akurasi di validation set.
Pangkas (hapus) node yang paling meningkatkan akurasi validation set.
- Berhenti ketika pemangkasan lebih lanjut bersifat merugikan (menurunkan akurasi).

2. Rule Post-Pruning (digunakan di C4.5)

Ini adalah peningkatan dari ID3 dan cocok untuk data yang terbatas.

- Tumbuhkan tree sampai fit (boleh overfit).
- Konversi tree menjadi rules (aturan). Setiap satu jalur dari root ke leaf menjadi satu rule. Contoh Rule: IF Outlook=Sunny AND Humidity=High THEN Play=No.
- Pangkas (generalisasi) setiap rule dengan menghapus prakondisi (misal, Humidity=High) jika penghapusan itu meningkatkan estimasi akurasi rule.
- Urutkan rules yang sudah dipangkas berdasarkan akurasinya. Gunakan urutan ini saat mengklasifikasikan data baru.

Mengapa perlu mengubah Decision Tree menjadi Decision Rules?

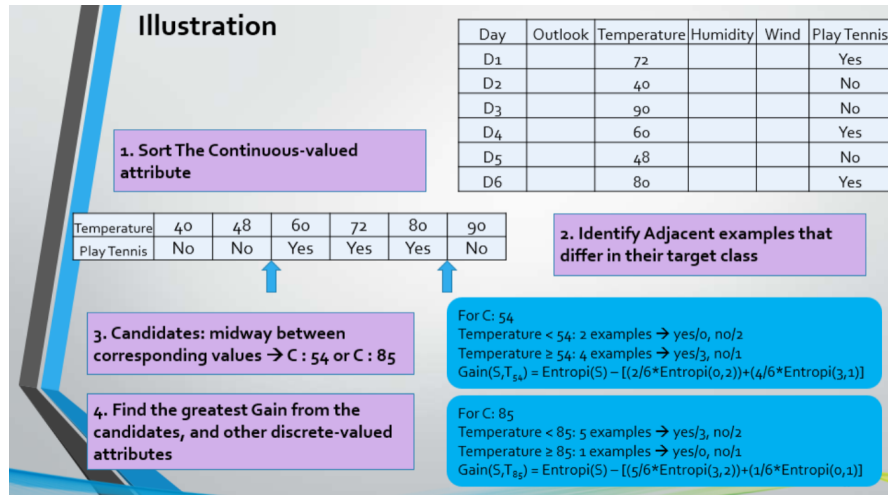
- Setiap rule (jalur) dapat dipangkas secara independen.
- Menghilangkan perbedaan antara tes atribut (misal, tes di dekat root vs. di dekat leaf diperlakukan sama).
- Meningkatkan keterbacaan (lebih mudah dipahami manusia).

Konsep: Continuous-valued Attribute

Masalah: Algoritma DTL standar (seperti ID3) dirancang untuk atribut diskrit (seperti Outlook=Sunny/Overcast/Rain). Bagaimana jika atributnya kontinu (seperti Suhu=80.5, 72.3, 90.1)?

Solusi: *Discretization* (Diskretisasi)

- Ide: Mengubah atribut kontinu (misal, Temperature) menjadi atribut boolean (diskrit) baru (misal, Temperature_54) berdasarkan sebuah threshold c .
- Cabang Baru: Tree akan memiliki dua cabang, misal: Temperature < 54 (True) dan Temperature >= 54 (False).



Gambar 4.9: Mekanisme *Discretization* untuk atribut kontinu (Sumber: Slide Issues with DTL, Hal 22)

Tahapan menentukan *threshold c* Terbaik:

1. Urutkan (Sort): Urutkan semua nilai atribut kontinu tersebut dari terkecil ke terbesar.
2. Identifikasi: Cari pasangan nilai yang berdekatan yang memiliki kelas target berbeda. (Misal: $Temp = 48$ (No), $Temp = 60$ (Yes)).
3. Buat Kandidat: Buat *threshold* kandidat c diengah-tengah pasangan tersebut. (Misal: $c = (48 + 60)/2 = 54$). Akan ada beberapa kandidat (misal, $c = 85$ juga kandidat).
4. Hitung Gain: Hitung Information Gain untuk setiap kandidat *threshold* seolah-olah itu adalah atribut diskrit.
5. Pilih: Pilih kandidat c (misal, $c = 54$ atau $c = 85$) yang memberikan *Gain* tertinggi. *Gain* ini kemudian dibandingkan dengan *Gain* dari atribut-atribut diskrit lainnya (seperti Outlook, Wind) untuk memilih *root* terbaik.

Notes

Perlu diperhatikan bahwa jika terdapat pasangan nilai yang berdekatan dengan kelas target berbeda dan salah satunya memiliki *missing value*, maka tidak akan dianggap sebagai *potensial optimal breakpoint*

Exercise

Outlook	Temp	Humidity	Windy	Class
sunny	75	70	TRUE	Play
sunny	80	90	TRUE	Don't Play
sunny	85	85	FALSE	Don't Play
sunny	72	95	FALSE	Don't Play
sunny	69	70	FALSE	Play
?	72	90	TRUE	Play
overcast	83	78	FALSE	Play
overcast	64	65	TRUE	Play
overcast	81	75	FALSE	Play
rain	71	80	TRUE	Don't Play
rain	65	70	TRUE	Don't Play
rain	75	80	FALSE	Play
rain	68	80	FALSE	Play
rain	70	96	FALSE	Play

1. What is GainRatio for Outlook?
2. What are Examples (instances) for Outlook = sunny and what is the weight for instance with outlook=? related with outlook=sunny
3. Based on the result of question (2), define the threshold in "Humidity" discretization, and the leaf node for each branch.

Gambar 4.10: Exercise Issues with DTL (Sumber: Slide Issues with DTL, Hal 35)

Konsep: Missing Attribute Value

Masalah: Apa yang harus dilakukan jika data tidak lengkap (misal Outlook=Sunny, Humidity=?) saat *training* atau *validation*?

Strategi Alternatif:

- Common Value (Node): Isi nilai yang hilang dengan nilai yang paling umum dari atribut tersebut di node saat ini.
- Common Value (Class): Isi dengan nilai paling umum dari atribut tersebut, khusus dari contoh-contoh di node saat ini yang memiliki klasifikasi yang sama.
- Probabilitas (C4.5): Tetapkan probabilitas untuk setiap kemungkinan nilai.

Tahapan untuk strategi probabilitas:

- Misal, di simpul n , atribut A punya 6 nilai 'v1' dan 4 nilai 'v2' (total 10 data).
- Untuk data ke-11 yang nilai A-nya missing, data ini akan "dibagi": 0.6 (6/10) dikirim ke cabang 'v1' dan 0.4 (4/10) dikirim ke cabang 'v2'.
- Saat menghitung $Gain(S, A)$ dan $Entropy(S)$, perhitungannya hanya mempertimbangkan fraksi contoh yang nilainya diketahui.

$$Entropy(S) = -\frac{6}{10} \log_2 \left(\frac{6}{10} \right) - \frac{4}{10} \log_2 \left(\frac{4}{10} \right)$$

dan

$$Gain(S, A) = \frac{10}{11} (E(S) - [\sum \text{proportion} \times \text{known_value_entropy}]) = \frac{10}{11} \left(E(S) - \left[\frac{6}{10} E(v_1) + \frac{4}{10} E(v_2) \right] \right)$$

Notes

Menganggap "Hilang" (misal, "?") sebagai nilai tersendiri tidak selalu tepat. Ini tidak boleh dilakukan jika nilai bisa hilang karena alasan yang berbeda. Contoh: Atribut **IsPregnant** yang missing untuk pasien Pria harus diperlakukan sebagai 'No', sedangkan jika missing untuk pasien Wanita usia 25, itu harus diperlakukan sebagai 'Unknown'.

Konsep: Alternative Measure for Selecting Attribute

Masalah: Information Gain (ukuran standar di ID3) memiliki bias. Ia sangat menyukai atribut yang memiliki banyak nilai unik (misal, Atribut 'Tanggal' atau 'ID Mahasiswa').

Kenapa Buruk? Atribut 'Tanggal' mungkin akan mengklasifikasikan data latihan dengan sempurna (Gain sangat tinggi), tapi prediktor yang sangat buruk untuk data baru. Ini adalah bentuk *overfitting*.

Solusi: Gain Ratio (digunakan di C4.5)

- Ide: Menormalkan Information Gain dengan membaginya dengan SplitInformation.
- SplitInformation: Ini adalah metrik yang mengukur seberapa banyak node terpecah. Nilainya akan sangat tinggi untuk atribut seperti 'Tanggal' (karena terpecah menjadi banyak sekali cabang, satu untuk setiap tanggal).
- Hasil: Atribut seperti 'Tanggal' akan memiliki SplitInformation yang tinggi, sehingga GainRatio-nya menjadi kecil ($\text{Gain} / \text{InfoSplit_Tinggi} = \text{Kecil}$). Ini "menghukum" atribut yang memiliki terlalu banyak nilai.

Dengan persamaan

$$\text{SplitInformation}(S, A) \equiv - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

dan

$$\text{GainRatio}(S, A) \equiv \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)}$$

Dalam hal ini:

S : himpunan data (dataset) awal.

A : atribut yang sedang dievaluasi.

S_i : subset dari S untuk nilai ke- i dari atribut A .

c : jumlah nilai berbeda (kategori) dari atribut A .

$|S|$: jumlah data dalam himpunan S .

$|S_i|$: jumlah data dalam subset S_i .

$\text{Gain}(S, A)$: Information Gain dari atribut A terhadap himpunan S .

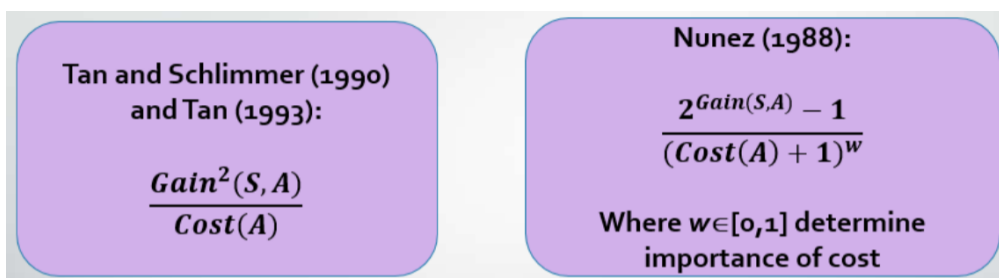
Masalah Gain Ratio ketika SplitInformation sangat kecil atau nol (misal, atribut hanya punya 1 nilai), GainRatio bisa menjadi tidak terdefinisi atau sangat besar. Solusinya (Heuristik), hanya hitung GainRatio untuk atribut-atribut yang memiliki Information Gain di atas rata-rata.

Konsep: Attribute with Differing Cost

Masalah: Di dunia nyata, mendapatkan data atribut tidak gratis. Beberapa atribut murah (misal, Temperature), sementara yang lain mahal (misal, BiopsyResult, BloodTestResults), baik secara moneter maupun kenyamanan pasien.

Tujuan: Decision tree harusnya lebih memilih atribut berbiaya rendah, dan hanya menggunakan atribut berbiaya tinggi jika benar-benar diperlukan untuk menghasilkan klasifikasi yang andal.

Solusi: Memasukkan Cost ke dalam formula perhitungan “Gain”. Tujuannya bukan lagi memaksimalkan Gain, tapi memaksimalkan (Gain / Cost).



Gambar 4.11: Contoh formula yang dapat digunakan sebagai Cost

Bab 5

Linear Models

- 5.1 Linear Regression 39
- 5.2 Logistic Regression 40
- 5.3 Proses Training Logistic Regression . . 41

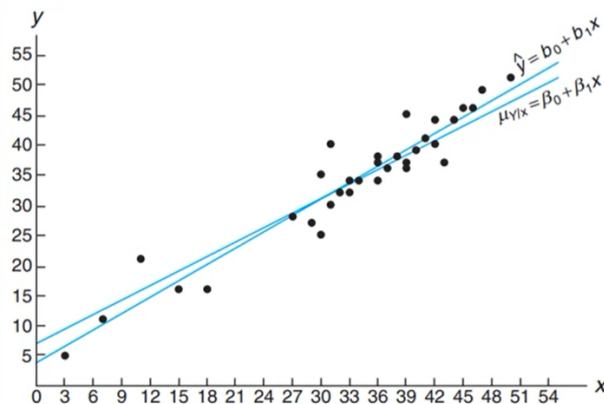
5.1 Linear Regression

Definisi 5.1

Regresi adalah kegiatan analisis untuk menemukan hubungan terbaik antara X dan Y . X didefinisikan sebagai *feature / independent variable / regressor*, sedangkan Y didefinisikan sebagai *target / dependent variable / response*.

Bentuk dari X dapat berupa vektor berdimensi d .

Analisis regresi bersifat nondeterministik dan random error.



Gambar 5.1: Visualisasi Regresi Linier

Pada analisis regresi, dibutuhkan data berlabel sehingga berupa pasangan nilai X dan Y . Hasil dari analisis regresi adalah fungsi target yang memetakan nilai X ke Y . Dalam regresi, yang dicari adalah estimasinya dalam bentuk $\hat{y} = b_0 + b_1 X$ sehingga yang diestimasi adalah b_0 dan b_1 .

Lantas, bagaimana caranya? Dengan mengkuantifikasi kekuatan hubungan antara X dengan Y . Ketika terdapat beberapa X , maka akan dicari X mana saja yang memiliki hubungan **linier** yang kuat dengan Y .

Perlu diperhatikan terdapat asumsi bahwa **random error** dengan ekspektasi 0 dan variasinya tetap (**homoscedasticity**) untuk bisa menggunakan metode Least Squared Estimation (LSE) yang menderivasi (turunan parsial) dari persamaan errornya sehingga dapat diperoleh b_0 dan b_1 yang meminimumkan error.

Notes

Sebenarnya pada IF3170, yang diujikan untuk regresi linier hanya cukup substitusi nilai-nilai X pada tiap datum untuk memperoleh Y . Tidak sampai mencari b_0 dan b_1 *from scratch* untuk mendapatkan fungsi target. (Pendekatan *linear regression + threshold*)

Contoh 5.1.1: UTS 2023

Diketahui hipotesis linear regression yang mewakili data latih (untuk klasifikasi) adalah

$$y = -0.313 + 0.051X_1 + 0.132X_2$$

dengan aturan jika $y < 0.5$, maka diprediksi sebagai kelas A (negatif), sebaliknya kelas B (positif).

Terdapat data latih dengan $X_1 = 2$ dan $X_2 = 4$. Tentukan kelas dari data latih tersebut.

5.2 Logistic Regression

Regresi Linear + Threshold memiliki beberapa kelemahan. Terdapat asumsi bahwa variansi harus konstan dan variabel target berdistribusi normal, sedangkan yang sering terjadi tidak normal. Kemudian, untuk dapat menggunakan LSE perlu banyak asumsi yang harus dipenuhi.

Definisi 5.2

Regresi Logistik adalah pendekatan yang lebih baik untuk klasifikasi. Regresi Logistik mengambil output dari model linear $z = b^T x = b_0 \cdot 1 + b_1 x_1 + \dots + b_d x_d$ dan memasukkannya ke dalam fungsi logistik (*sigmoid*) yang didefinisikan sebagai

$$p = P(y = 1|x, b) = \frac{1}{1 + e^{-b^T x}} \frac{1}{1 + e^{-z}}$$

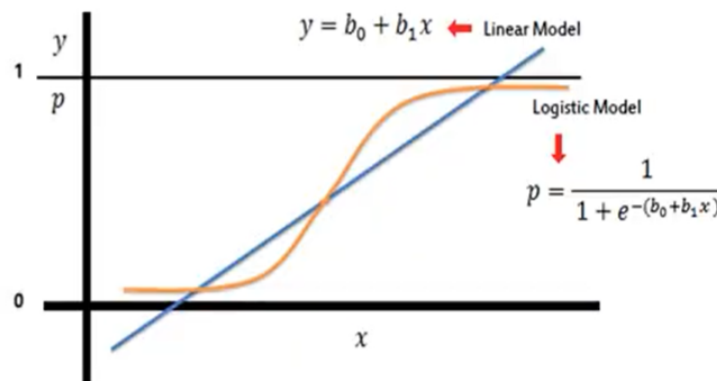
Fungsi ini “memaksa” output apapun (dari $-\infty$ sampai $+\infty$) menjadi nilai antara 0 dan 1, yang bisa diinterpretasikan sebagai probabilitas.

Jika $p \geq 0.5$, diprediksi kelas 1. Jika $p < 0.5$, diprediksi kelas 0.

Secara semantik, nilai $\hat{y}$ pada logistic regression menyatakan Log Odds dari peluang sukses, yaitu peluang dari variabel target yang bernilai 1.

$$y = \log(\text{Odds}) = \log\left(\frac{p}{1+p}\right) = b^T x$$

Sebagai *summary*, logistic regression merupakan model linier yang merupakan klasifikasi biner dan dirancang secara natural untuk *soft classification*, artinya *output* yang dihasilkan merupakan peluang untuk tiap kelas.



Gambar 5.2: Visualisasi Regresi Logistik

Notes

Sama halnya dengan regresi linear, untuk soal bertipe logistic regression sederhana cukup substitusi nilai-nilai X pada tiap datum untuk memperoleh Y . Namun, terdapat variasi soal yang perlu mencari *weights* yang berupa vektor $\mathbf{b}$ sebagai model logistic regression tersebut yang akan dibahas pada halaman berikutnya.

5.3 Proses Training Logistic Regression

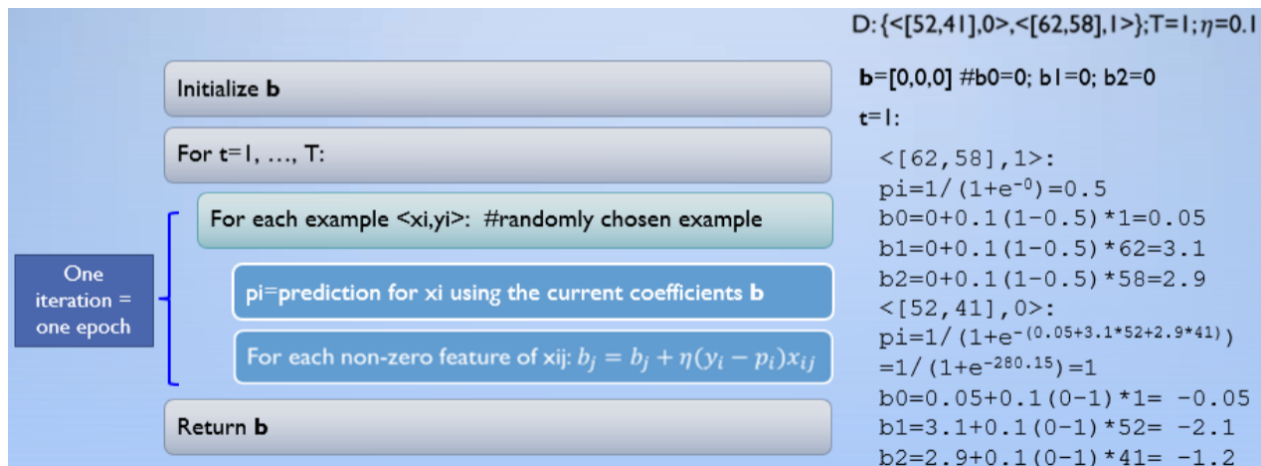
Definisi 5.3: Maximum Likelihood Estimator

Pelatihan logistic regression menggunakan pendekatan Maximum Likelihood Estimator. Yang dilakukan adalah menghitung terlebih dahulu LCL tiap datum / *instance* sehingga peluang dari label yang benar itu tertinggi ketika dijumlahkan menjadi maksimum.

$$LCL = \sum_{i=1}^n \log L(\theta; y_i | x_i) = \sum_{i=1; y_i=1}^n \log p_i + \sum_{i=1; y_i=0}^n \log(1 - p_i) \rightarrow \frac{\partial LCL}{\partial b_j} = \sum_i (y_i - p_i) x_{ij}$$

Untuk mencari **Maximum Likelihood Estimator**, digunakan **Stochastic Gradient Ascent** yang merupakan Local Search dengan inisialisasi sampel random kemudian dicari koefisien yang bisa meningkatkan nilai Log Likelihood. Proses *update* bobot pada tiap iterasi/*epoch* dihitung melalui dengan

$$b_j = b_j + \eta(y_i - p_i) x_{ij}$$



Gambar 5.3: Visualisasi Pelatihan Regresi Logistik

Contoh 5.3.1: UTS 2023

Seorang mahasiswa diminta melakukan proses pembelajaran dengan Logistic Regression untuk mendapatkan bobot w_0, w_1, w_2 sebanyak 1 iterasi. Gunakan: bobot awal w_0 s.d w_2 sebesar 0, nilai bias 1, learning rate 0.3. Urutan instance pembelajaran hasil pemilihan random adalah 3, 1, 4, dan 2.

- Tuliskan langkah pelatihan model dengan lengkap disertai rumus dan cara perhitungan. Gunakan ketelitian minimal 3 angka di belakang koma. Tuliskan bobot akhir hasil pelatihan. (Nilai 15)
- Hitung akurasi model pelatihan. (Nilai 5)

Notes

Model dari Logistic Regression yang dihasilkan adalah koefisien dari $\mathbf{b} \in \mathbb{R}^{d+1}$ yang dapat didefinisikan juga sebagai vektor $\mathbf{b} = b_0 + b_1x_1 + \dots + b_dx_d$ atau $\mathbf{b} = [b_0, b_1, \dots, b_d]$

Bab 6

Support Vector Machine

6.1	What is SVM	43
6.2	<i>Linear Separability</i> dan <i>Linear Classifier</i>	43
6.3	SVM Objective	44
6.4	SVM for Linearly Separable Data . . .	45
6.5	SVM for Non-Linearly Separable Data	46
6.6	Multi-Class Classification dengan SVM	48

6.1 What is SVM

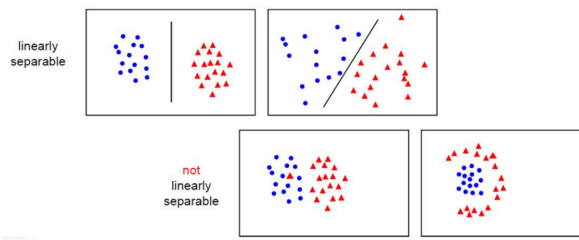
Definisi 6.1: Support Vector Machine

Support Vector Machine (SVM) adalah algoritma supervised learning untuk klasifikasi (dan regresi) yang diperkenalkan pada tahun 1992 oleh Vapnik, Boser, & Guyon.

- Era 1980-an (Neural Networks): NN populer karena bisa menangani data non-linear. Namun, NN memiliki kelemahan teoretis: bersifat heuristic/greedy dan rentan terjebak di local minima (solusi yang “cukup baik” tapi bukan yang terbaik).
- Era 1990-an (SVM): SVM muncul dengan landasan Teori Pembelajaran Komputasi (Computational Learning Theory) yang kuat. Keunggulan utamanya adalah penggunaan optimasi kuadrat (Quadratic Programming), yang menjamin ditemukannya global optimum (solusi matematika terbaik yang mutlak).

6.2 *Linear Separability* dan *Linear Classifier*

SVM fokus pada pemisahan data menjadi dua kelas (+1 dan -1). **Linear Separability** adalah kondisi di mana data dari dua kelas yang berbeda dapat dipisahkan secara sempurna oleh sebuah garis lurus (pada 2D) atau bidang datar (pada dimensi tinggi).



Gambar 6.1: Visualisasi Linearly Seperable dan Bukan Linearly Seperable

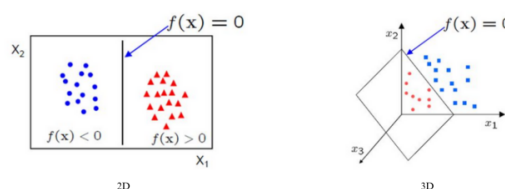
Definisi 6.2: Linear Classifier

Didefinisikan sebagai:

$$f(x) = w^T x + b$$

dengan w : normal terhadap garis, dikenal sebagai vektor bobot, b : bias

Dalam 2D, diskriminannya berupa sebuah garis. Dalam 3D, diskriminannya berupa sebuah bidang dan dalam n D berupa sebuah *hyperplane*



Gambar 6.2: Visualisasi Linear Classifier 2D dan 3D

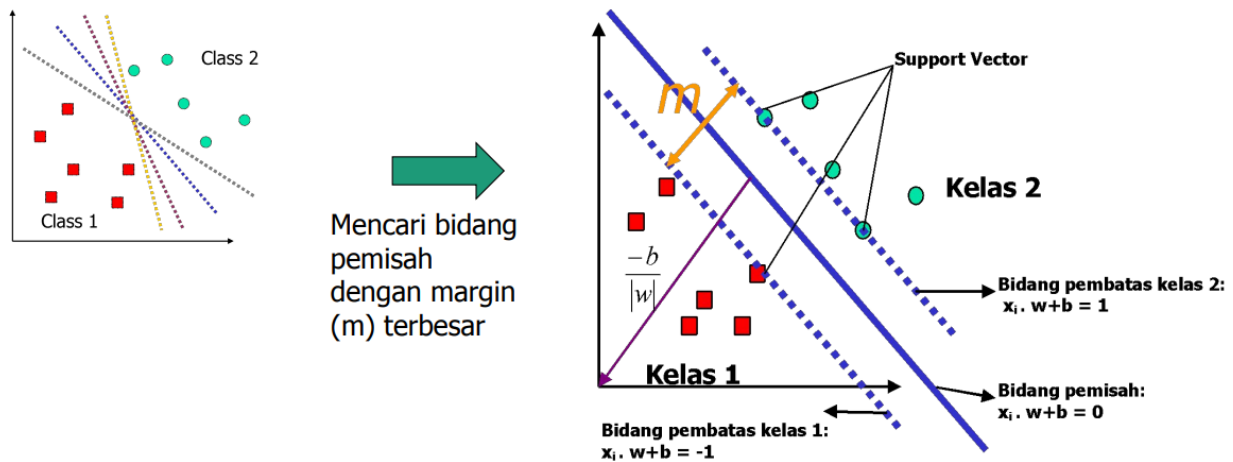
6.3 SVM Objective

Konsep

SVM bertujuan mencari Optimal Separating Hyperplane, yaitu pemisah yang memiliki Margin Terbesar. Menggunakan *quadratic programming* untuk menghindari *local minimum* dan fungsi kernel untuk memisahkan area yang nonlinier.

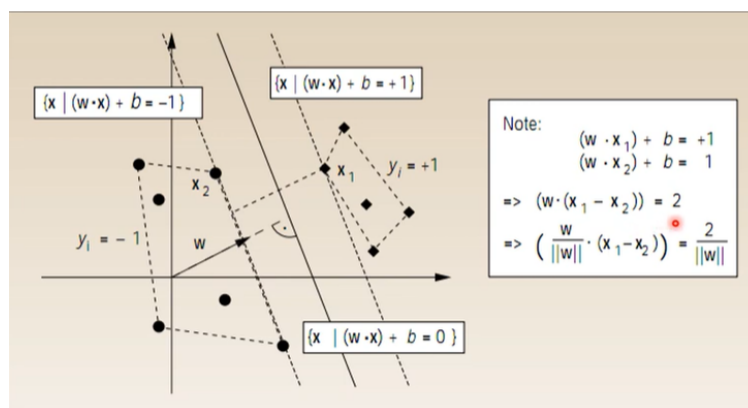
Margin: Jarak tegak lurus antara hyperplane dengan titik data terdekat dari masing-masing kelas.

Filosofi: “Zona aman” yang lebar (margin besar) memberikan toleransi kesalahan yang lebih baik saat menghadapi data baru yang belum pernah dilihat sebelumnya.



Gambar 6.3: Visualisasi Bidang Pemisah Terbaik

Batas keputusan dalam SVM didefinisikan dengan **Hyperplane**.



Gambar 6.4: Visualisasi Hyperplane

6.4 SVM for Linearly Separable Data

Definisi 6.3: Support Vectors

Titik-titik data yang berada paling dekat dengan Hyperplane (tepat di pinggir margin). Data lain yang jauh dari margin tidak berpengaruh pada model.

Support vectors memenuhi:

$$|\vec{w} \cdot \vec{x}_i + b| = 1$$

Untuk itu, semua data pelatihan dengan label “+” dan “-” juga memenuhi:

$$y_i (\vec{w} \cdot \vec{x}_i + b) \geq 1$$

Definisi 6.4: Optimal Hyperplane

Tujuan SVM adalah memaksimalkan lebar margin (m). Rumus Lebar Margin adalah jarak tegak lurus antara pembatas kelas +1 dan -1 adalah:

$$\text{Margin} = ||w||_2$$

Masalah Optimasi: Untuk memaksimalkan margin, harus meminimalkan $||w||$. Secara matematis, untuk memudahkan penurunan rumus (agar bisa diturunkan/derivative), meminimalkan kuadratnya. Primal Problem (Masalah Asli):

$$\text{Minimize} : \frac{1}{2} ||w||^2 \quad \text{Subject to: } y_i \cdot (w \cdot x_i + b) \geq 1$$

Semua data harus benar di luar margin. Ini adalah masalah Convex Quadratic Programming (QP).

Definisi 6.5: Lagrange Multipliers dan Dual Problem

Masalah optimasi di atas (Primal) sulit diselesaikan langsung karena ada kendala pertidaksamaan. Perlu mengubahnya menjadi masalah Dual menggunakan metode Lagrange Multipliers.

Transformasi ke Dual Problem dengan diperkenalkan variabel baru bernama Alpha (α_i) untuk setiap titik data. Fungsi Dual yang Harus Dimaksimalkan:

$$\text{Maximize} : \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

Subject to:

1. $\sum \alpha_i y_i = 0$
2. $\forall i \in [1 \dots n] : \alpha_i \geq 0$

Support Vectors:

$$\alpha_i > 0$$

Hyperplane:

$$\vec{w} \cdot \vec{x} = \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \cdot \vec{x} = \sum_{i=1}^n \alpha_i y_i (\vec{x}_i \cdot \vec{x}) ; \quad b = y_{sv} - \vec{w} \cdot \vec{x}_{sv}$$

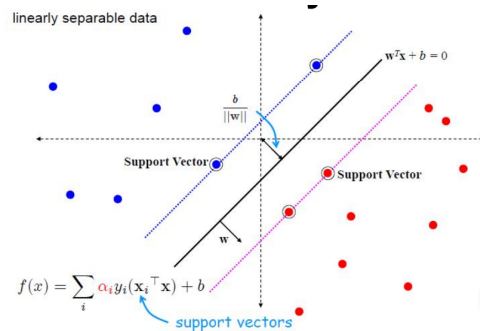
Notes

Persamaan yang dipakai ujung-ujungnya adalah:

$$f(\vec{x}) = \sum_{i=1}^{n_{sv}} (\alpha_i y_i \vec{x}_i \cdot \vec{x}) + b$$

dan

$$\sum \alpha_i y_i = 0$$



Gambar 6.5: Visualisasi SVM pada Linearly Seperatable Data

6.5 SVM for Non-Linearly Separable Data

Konsep

Dalam dunia nyata, data jarang sekali terpisah sempurna secara linear. Hal ini disebabkan oleh dua faktor utama:

Noise: Data outlier atau kesalahan pengukuran yang membuat pemisahan sempurna tidak mungkin dilakukan tanpa overfitting.

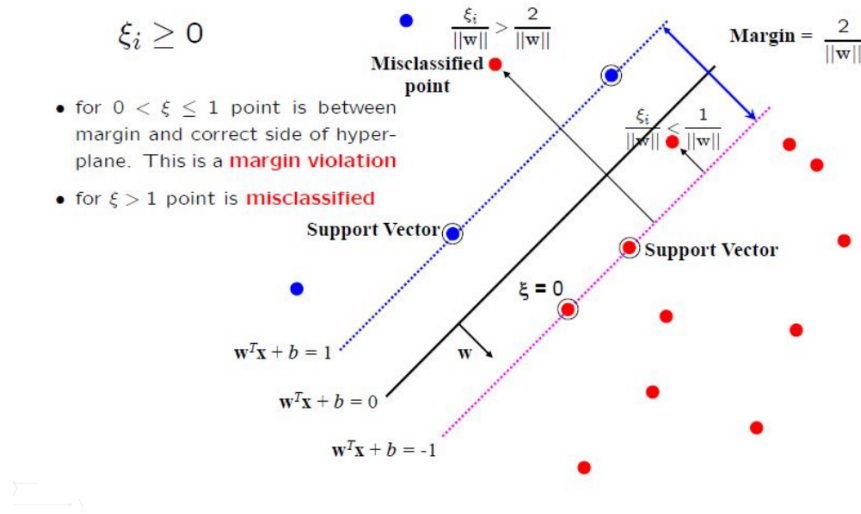
Sifat Alami Data: Pola data memang kompleks (misalnya melingkar) sehingga batasnya bukan garis lurus.

Definisi 6.6: Solusi 1: Slack Variable (C)

Untuk menangani noise, SVM memodifikasi kendala (constraint) agar lebih longgar. Kita mengizinkan beberapa data salah diklasifikasikan atau masuk ke dalam margin.

Slack Variable ($\xi_i = 0$) adalah Variabel non-negatif yang mengukur seberapa jauh sebuah titik melanggar margin.

- $\xi_i = 0$: Data terklasifikasi benar dan di luar margin (aman).
- $0 < \xi \leq 1$: Data berada di dalam margin tetapi masih di sisi yang benar.
- $\xi_i > 1$: Data salah diklasifikasikan (misclassified).



Gambar 6.6: Visualisasi Slack Variable

Fungsi Objektif Baru: meminimalkan $\|w\|^2$ dan meminimalkan total kesalahan ($\sum \xi_i$).

$$\text{Minimize: } \frac{1}{2} \|\vec{w}\|^2 + C \sum_{i=1}^N \xi_i$$

Formulasi Dual Baru:

$$Q(\vec{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \vec{x}_i^T \vec{x}_j$$

Subject to:

$$0 \leq \alpha_i \leq C$$

Peran Parameter C (Regularization):

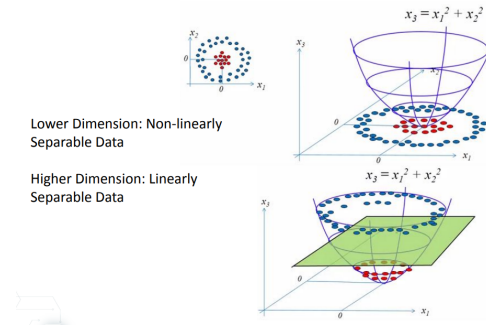
- C Besar (Strict): Memberi hukuman berat pada setiap ξ . Akibat: Margin sempit, potensi **Overfitting**.
- C Kecil (Tolerant): Lebih santai terhadap error. Mengutamakan margin yang lebar. Akibat: Margin lebar, lebih **robust** (potensi *Underfitting* jika terlalu kecil).

Definisi 6.7: Solusi 2: Non-Linear Boundary Transformation

Jika data memiliki pola non-linear (misalnya: lingkaran), garis lurus tidak akan pernah bisa memisahkannya.

Ide Utama: Petakan data dari dimensi rendah (**Input Space**) ke dimensi yang lebih tinggi (**Feature Space**) menggunakan fungsi pemetaan $\phi(\vec{x})$.

- **Contoh:** Data 2D melingkar $(\vec{x}_1, \vec{x}_2)$ dipetakan ke 3D $(\vec{z}_1, \vec{z}_2, \vec{z}_3)$ di mana $\vec{z}_3 = \vec{x}_1^2 + \vec{x}_2^2$.
- Di dimensi 3D, data tersebut mungkin menjadi terpisah secara linear oleh sebuah bidang datar.
- Saat dikembalikan ke 2D, bidang datar itu menjadi garis lengkung (lingkaran).



Gambar 6.7: Transformasi pada Non-Linearly Separable Data

Definisi 6.8: The Kernel Trick

Menghitung transformasi $\phi(\vec{x})$ ke dimensi sangat tinggi (bahkan tak hingga) itu mahal dan lambat.

- **Triknnya:** Dalam optimasi Dual Problem, kita hanya butuh hasil Dot Product $(\vec{x}_i \cdot \vec{x}_j)$. Kita tidak butuh koordinat aslinya.
- Maka, kita definisikan **Fungsi Kernel** $K(\vec{x}_i, \vec{x}_j)$ yang bisa langsung menghitung hasil dot product di dimensi tinggi TANPA kita perlu tahu bentuk transformasi $\phi(\vec{x})$ -nya.

$$K(\vec{x}_i, \vec{x}_j) = \phi(\vec{x}_i) \cdot \phi(\vec{x}_j)$$

Jadi, Fungsi Keputusan Akhir menjadi:

$$f(\vec{x}) = \sum_{i=1}^{n_{sv}} \alpha_i y_i K(\vec{x}_i, \vec{x}) + b$$

Kernel	Rumus Matematika	Karakteristik
Linear	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$	Sama seperti SVM biasa. Cepat. Bagus untuk data teks/dimensi sangat tinggi.
Polynomial	$K(\mathbf{x}, \mathbf{z}) = (\gamma \mathbf{x}^T \mathbf{z} + r)^d$	Membentuk batas melengkung polinomial. Parameter d (derajat) menentukan kompleksitas.
RBF (Gaussian)	$K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$	-
Sigmoid	$K(\mathbf{x}, \mathbf{z}) = \tanh(\gamma \mathbf{x}^T \mathbf{z} + r)$	Mirip perilaku Neural Network.

Tabel 6.1: Jenis-Jenis Kernel Populer

6.6 Multi-Class Classification dengan SVM

Konsep

Secara desain, SVM adalah binary classifier (klasifikasi biner). Algoritma ini hanya mengerti cara memisahkan dua hal: kelas positif (+1) dan kelas negatif (-1).

Untuk mengklasifikasikan data yang memiliki lebih dari dua kelas (misalnya: apel, jeruk, mangga), kita harus memecah masalah tersebut menjadi beberapa sub-masalah biner.

Definisi 6.9: Strategi One-against-All (OvA) / One-against-Rest

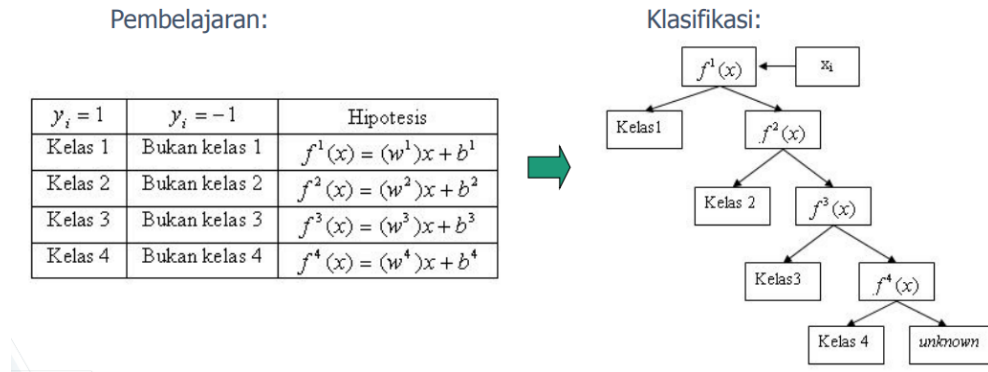
Strategi ini membangun satu model untuk setiap kelas, yang memisahkan kelas tersebut dari semua kelas lainnya.

Jumlah Model: Jika ada k kelas, kita membangun k model SVM.

Konsep:

- Model 1: Kelas A vs (B + C + D...)
- Model 2: Kelas B vs (A + C + D...)
- dst.

Keputusan (Klasifikasi): Untuk data baru x , kita masukkan ke semua k model. Pemenangnya adalah kelas yang memberikan nilai fungsi keputusan $f(x)$ terbesar (paling positif/yakin).



Gambar 6.8: Multiclass SVM dengan OVA

Definisi 6.10: One-against-One (OvO)

Strategi ini membangun satu model untuk setiap pasangan kelas.

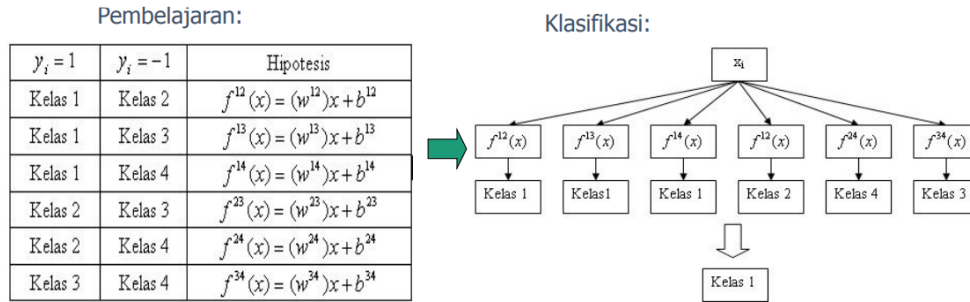
Jumlah Model: Kita membangun $\frac{k(k-1)}{2}$ model SVM.

- Contoh: Jika ada 4 kelas, butuh $\frac{4(3)}{2} = 6$ model.

Konsep:

- Model 1: Kelas A vs Kelas B
- Model 2: Kelas A vs Kelas C
- Model 3: Kelas B vs Kelas C
- dst.

Keputusan (Klasifikasi): Menggunakan metode Max Wins (Voting). Data dimasukkan ke semua model. Jika Model 1 memprediksi A, maka A dapat 1 poin. Kelas dengan jumlah poin terbanyak di akhir adalah pemenangnya.



Gambar 6.9: Multiclas SVM dengan OVO

Definisi 6.11: DAGSVM (Directed Acyclic Graph SVM)

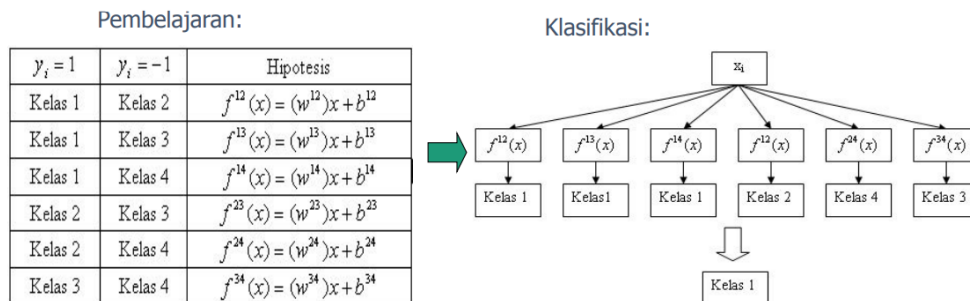
Metode ini menggunakan model-model dari strategi One-against-One (OvO), tetapi cara pengambilan keputusannya berbeda untuk mempercepat proses testing.

Struktur: Menggunakan struktur graf/pohon (Directed Acyclic Graph).

Proses (Turnamen Eliminasi): Dimulai dari akar (misal: A vs D).

- Jika hasilnya A, maka D dieliminasi (D tidak mungkin benar). Kita bergerak ke simpul yang menguji A vs kandidat lain.
- Jika hasilnya D, maka A dieliminasi.

Keuntungan: Lebih cepat saat testing karena tidak perlu mengevaluasi semua $\frac{k(k-1)}{2}$ model, melainkan hanya menyusuri jalur graf (sekitar $k - 1$ evaluasi).



Gambar 6.10: Multiclas SVM dengan DAGSVM

Konsep: Library Populer untuk SVM

LibSVM: Library paling populer (basis algoritma kernel SVM di Scikit-Learn). Menggunakan strategi One-against-One untuk multi-class. Mendukung C++, Java, Python.

LibLinear: Versi optimasi khusus untuk Linear Kernel. Sangat cepat untuk data dimensi tinggi (seperti teks). Menggunakan strategi One-against-All.

SVMLight: Populer untuk klasifikasi teks.

Bab 7

Bayes Net

7.1	Introduction to Probabilistic Reasoning System	52
7.2	Independence dalam Bayesian Network	53
7.3	Inference dalam Bayesian Network . .	55
7.4	Learning Bayesian Network dari Data	58

7.1 Introduction to Probabilistic Reasoning System

Konsep

PRS adalah bentuk Supervised Learning dengan ketidakpastian (Non-Deterministic).

Fungsi: Menghitung peluang kejadian x berdasarkan kondisi/bukti e yaitu $P(x|e)$.

Dasar: Teori probabilitas sebagai model "derajat kepercayaan" (degree of belief).

Definisi 7.1: Full Joint Probability Distribution

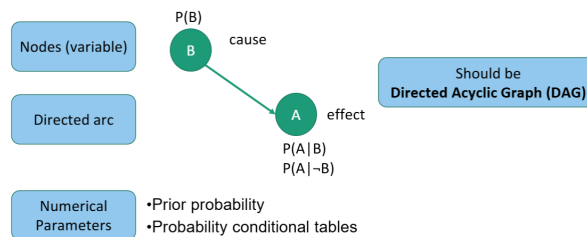
Untuk merepresentasikan dunia dengan N variabel biner, distribusi probabilitas gabungan (Full Joint Probability Distribution) membutuhkan 2^N nilai probabilitas.

- Jika $N = 5$, butuh $2^5 = 32$ entri.
- Jika N besar, ukuran tabel menjadi *intractably large* (terlalu besar untuk dikelola).

Konsep: Solusi: Bayesian Network (BN)

BN (disebut juga *Belief Nets* atau *Causal Networks*) memanfaatkan hubungan independensi antar variabel untuk mengurangi jumlah probabilitas yang perlu disimpan secara drastis.

Contoh: Untuk $N = 5$, Full Joint butuh 32 nilai, BN dengan struktur kausal mungkin hanya butuh 20 nilai.



Gambar 7.1: Struktur Bayesian Network

Definisi 7.2: Komponen Bayesian Network

BN merepresentasikan ketergantungan kausal secara grafis.

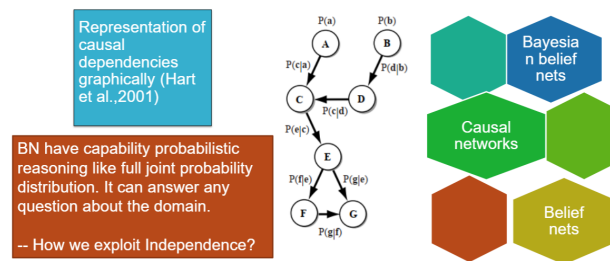
Struktur: Harus berupa DAG (Directed Acyclic Graph). Tidak boleh ada cycle (loop).

Nodes: Merepresentasikan variabel kejadian (misal: Burglary, Earthquake).

Arcs (Panah): Merepresentasikan hubungan sebab-akibat (causal dependencies). Arah panah dari *Cause* ke *Effect*.

Numerical Parameters:

- Untuk node tanpa orang tua (root): Probabilitas Prior ($P(A)$).
- Untuk node dengan orang tua: Tabel Probabilitas Kondisional / CPT ($P(\text{Effect}|\text{Cause})$).



Gambar 7.2: Topologi Bayesian Network

Konsep: Topologi Jaringan

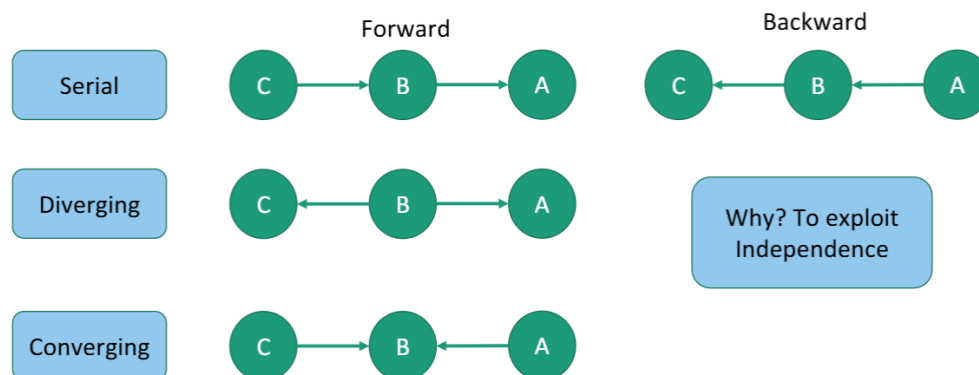
Topologi menentukan asumsi independensi. Node X dipengaruhi langsung oleh Parents-nya.

BN meng-*encoding* informasi: "Diberikan parents-nya, sebuah node independen terhadap non-descendants-nya".

7.2 Independence dalam Bayesian Network

Konsep: Tipe Koneksi & Aliran Informasi

Untuk menentukan apakah dua variabel independen atau dependen, kita melihat jalur (path) dan status pengetahuannya (evidence).



Gambar 7.3: Tipe Koneksi dalam Bayesian Network

Definisi 7.3: Serial (Linear): $A \rightarrow B \rightarrow C$

Informasi mengalir dari A ke C melalui B .

- **Jika B TIDAK diketahui:** A dan C saling mempengaruhi (dependent). Mengetahui A memberi petunjuk tentang B , yang memberi petunjuk tentang C .
- **Jika B DIKETAHUI (Instantiated):** Hubungan terputus. A dan C menjadi Conditionally Independent. B memblokir aliran informasi.

Definisi 7.4: Diverging (Common Cause): $A \leftarrow B \rightarrow C$

B adalah penyebab umum A dan C .

- **Jika B TIDAK diketahui:** A dan C berkorelasi (dependent). Jika A terjadi, kemungkinan B terjadi meningkat, sehingga kemungkinan C terjadi juga meningkat.
- **Jika B DIKETAHUI:** Hubungan terputus. A dan C menjadi Conditionally Independent. Mengetahui penyebab (B) memisahkan efek-efeknya.

Definisi 7.5: Converging (Common Effect / V-Structure): $A \rightarrow B \leftarrow C$

A dan C adalah penyebab independen dari B .

- **Jika B TIDAK diketahui:** A dan C Independen. Informasi A tidak memberitahu apa-apa tentang C .
- **Jika B (atau turunannya) DIKETAHUI:** A dan C menjadi DEPENDENT. Ini disebut "*Explaining Away*".

Contoh: A = Hujan, C = Sprinkler, B = Rumput Basah.

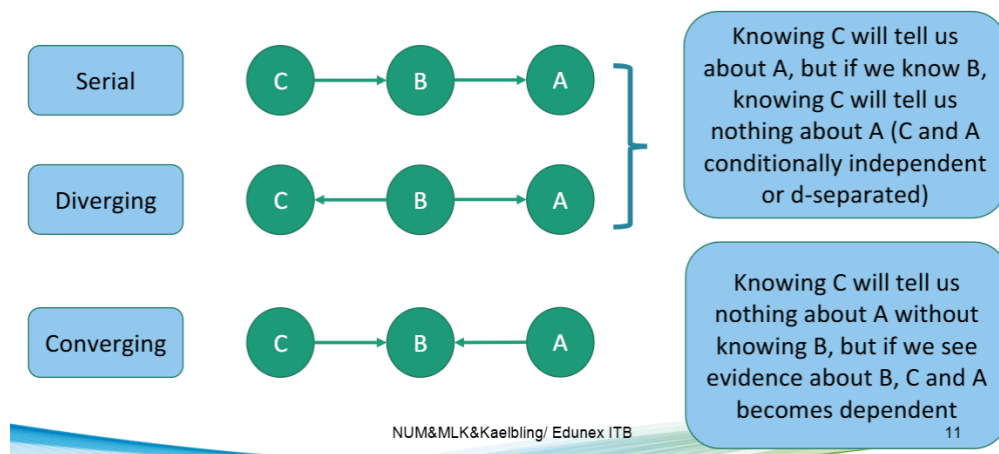
Jika kita tahu Rumput Basah ($B = \text{True}$), dan kita tahu Hujan ($A = \text{True}$), maka peluang Sprinkler ($C = \text{True}$) turun (karena hujan sudah menjelaskan basahanya rumput).

Definisi 7.6: D-Separation (Direction-dependent Separation)

Dua variabel *d-separated* (independen) jika SEMUA jalur di antara mereka terblokir.

Sebuah jalur terblokir jika ada node perantara V di mana:

1. Koneksi Serial/Diverging DAN V diketahui.
2. Koneksi Converging DAN V (maupun turunannya) TIDAK diketahui.



Gambar 7.4: Direction-dependent Separation dalam Bayesian Network

Contoh Kasus: Holmes & Watson

Konsep: Struktur Jaringan

- Burglary $\rightarrow$ Alarm $\leftarrow$ Earthquake (Converging di Alarm).
- Alarm $\rightarrow$ WatsonCalls & Alarm $\rightarrow$ MaryCalls (Diverging dari Alarm).

Definisi 7.7: Analisis Independence

- Jika Alarm tidak berbunyi (tidak diketahui), Burglary dan Earthquake independen.
- Jika Alarm berbunyi (diketahui), Burglary dan Earthquake menjadi dependen (*Explaining Away*).
- WatsonCalls dan MaryCalls dependen satu sama lain, KECUALI jika status Alarm diketahui pasti, maka mereka jadi independen.

7.3 Inference dalam Bayesian Network

Definisi 7.8: Konsep Inference

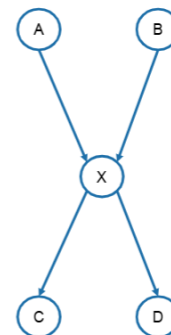
Inference adalah proses menghitung probabilitas posterior dari variabel Query (Q) diberikan variabel Evidence (e).

Target: $P(Q|e)$.

Chain Rule BN: Probabilitas gabungan dalam BN adalah perkalian probabilitas setiap node diberikan parents-nya:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$$

- X represents the fish : x1=salmon and x2=sea bass.
- X is influenced by A and B.
- A represents time of year: a1 = winter, a2 = spring, a3 = summer and a4 = autumn. Probability distribution on A in uniform.
- B represents geographical area where the fish was caught: b1 = north Atlantic and b2 = south Atlantic. The probabilities that any fish came from those areas are 0.6 and 0.4.
- C represents lightness with c1 = light, c2 = medium and c3 = dark
- D represents thickness with d1 = wide and d2 = thin.



Gambar 7.5: Contoh kasus inferensi dalam Bayesian Network

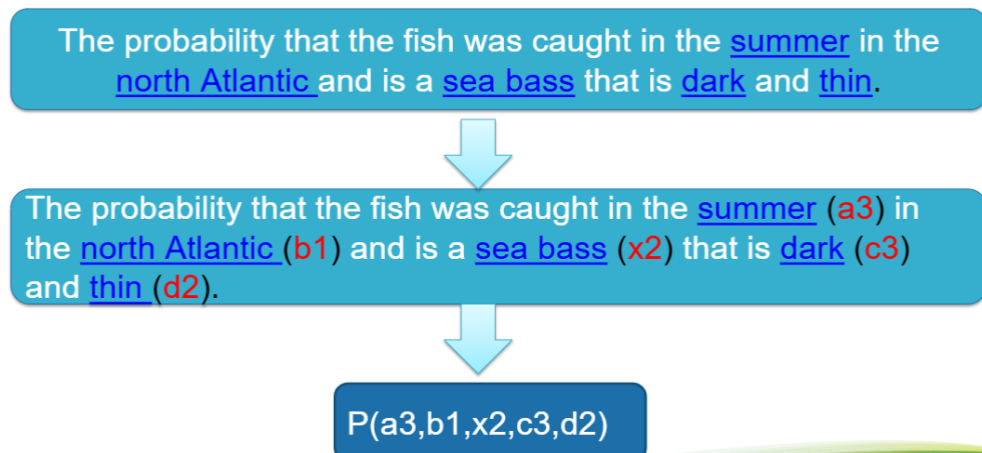
Konsep: Kasus: Ikan (Fish)

Variables: A (Time), B (Loc), X (Fish Type: Salmon/Seabass), C (Lightness), D (Thickness).

Query: $P(a_3, b_1, x_2, c_3, d_2)$

Hitung: $P(a_3)P(b_1)P(x_2|a_3, b_1)P(c_3|x_2)P(d_2|x_2)$.
(Ambil nilai dari CPT yang tersedia).

$$= 0.25 \times 0.6 \times 0.6 \times 0.5 \times 0.4 = 0.012$$



Gambar 7.6: Query inferensi dalam Bayesian Network

Definisi 7.9: Classification dalam BN

Klasifikasi adalah bentuk inferensi spesifik di mana kita ingin menentukan kelas (X) berdasarkan atribut yang diamati (e).

Tujuan: Mencari MAP (Maximum A Posteriori). Bandingkan $P(x_1|e)$ vs $P(x_2|e)$.

Rumus:

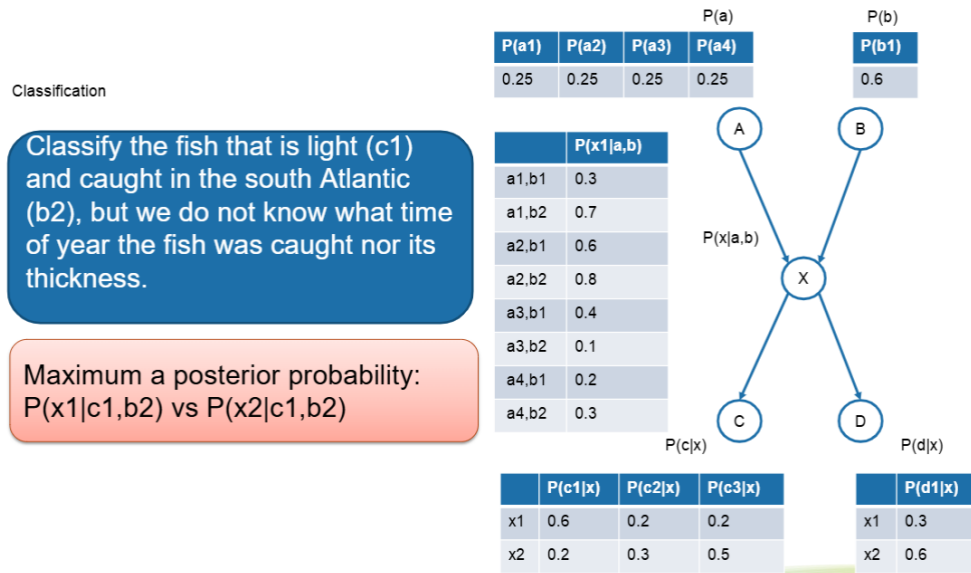
$$P(X|e) = \frac{P(X, e)}{P(e)} = \alpha P(X, e) = \alpha \sum_y P(X, e, y)$$

Dimana y adalah variabel hidden (tidak diketahui) yang harus dijumlahkan (summed out). α adalah konstanta normalisasi ($1/P(e)$).

Konsep: Langkah Klasifikasi (Contoh Ikan)

Diketahui: Light (c_1), South Atlantic (b_2). Apa jenis ikannya (X)?

Target: Bandingkan $P(x_1|c_1, b_2)$ dan $P(x_2|c_1, b_2)$.



Gambar 7.7: Langkah inferensi dalam Bayesian Network

Contoh 7.3.1: Klasifikasi Ikan dengan Bayesian Network

Diketahui: Light (c_1), South Atlantic (b_2). Tentukan jenis ikannya (X).

Target: Bandingkan $P(x_1|c_1, b_2)$ dan $P(x_2|c_1, b_2)$.

Jawab:

1. **Hitung untuk x_1 (Salmon):**

Kita harus menjumlahkan semua kemungkinan variabel A (Time) dan D (Thickness) karena tidak diketahui.

$$P(x_1, c_1, b_2) = \sum_A \sum_D P(A)P(b_2)P(x_1|A, b_2)P(c_1|x_1)P(D|x_1)$$

Karena $\sum P(D|x_1) = 1$ (karena x parent D), variabel D bisa diabaikan dalam penjumlahan. Fokus pada penjumlahan terhadap A :

$$= P(b_2)P(c_1|x_1) \times \left[\sum_{i=1}^4 P(a_i)P(x_1|a_i, b_2) \right]$$

$$= \text{Konstanta} \times \text{Hasil Sum } A = 0.114 \text{ (misal).}$$

2. **Hitung untuk x_2 (Sea bass):**

Lakukan hal sama untuk x_2 . Misal hasil = 0.042.

3. **Normalisasi & Keputusan:**

Total = 0.114 + 0.042 = 0.156.

- $P(x_1|c_1, b_2) = 0.114/0.156 = 0.73$.
- $P(x_2|c_1, b_2) = 0.042/0.156 = 0.27$.

Keputusan: x_1 (Salmon) karena memiliki probabilitas posterior lebih tinggi.

7.4 Learning Bayesian Network dari Data

Definisi 7.10: Estimasi Parameter dari Dataset D

Jika struktur BN sudah diketahui, kita tinggal menghitung probabilitas dari dataset D .

Node tanpa parent (V_i):

$$P(V_i = T) \approx \frac{\text{Jumlah kasus } V_i = T}{\text{Total data } k}$$

Node dengan parent (V_i punya parent V_j):

$$P(V_i = T | V_j = T) \approx \frac{\text{Jumlah kasus } V_i = T \text{ DAN } V_j = T}{\text{Jumlah kasus } V_j = T}$$

Definisi 7.11: Laplace Smoothing (Menangani Zero Probability)

Jika dalam data latih sebuah kejadian tidak pernah muncul (count = 0), probabilitasnya menjadi 0. Ini berbahaya karena jika dikalikan dalam Chain Rule, akan menolak seluruh prediksi.

Solusi: Laplace Smoothing (+1).

Menambahkan angka 1 pada pembilang dan menyesuaikan penyebut.

$$P(V_i = T) \approx \frac{\#(V_i = T) + 1}{k + 2}$$

(Penyebut +2 karena variabel biner punya 2 kemungkinan: T/F).

Konsep: Constructing BN Structure (Structure Learning)

Bagaimana jika struktur graf belum ada?

1. Tentukan himpunan variabel.
2. Tentukan urutan variabel.
3. Tambahkan node satu per satu, dan tentukan parents-nya.

Definisi 7.12: Pendekatan Structure Learning

Causal Knowledge: Menggunakan logika sebab-akibat (Misal: Hujan menyebabkan Basah, bukan sebaliknya).

Data-Driven: Menggunakan tes independensi statistik pada data.

- Cek apakah $P(C|A) = P(C)$? Jika ya, A bukan parent C .
- Cek apakah $P(C|A, B) = P(C|B)$? Jika ya, A tidak relevan jika B diketahui.

Pentingnya Urutan: Urutan penambahan node yang berbeda dapat menghasilkan struktur jaringan yang berbeda (dan mungkin lebih kompleks/kurang efisien). Urutan terbaik adalah Cause $\rightarrow$ Effect.

Bab 8

Unsupervised Learning (Clustering)

8.1	Reviu Unsupervised Learning	60
8.2	Apa itu Clustering?	60
8.3	Tahapan Proses Clustering	62
8.4	Representasi Cluster	62
8.5	Taksonomi Metode Clustering	63
8.6	Partitioning Method: K-Means Clustering	64
8.7	Density-Based Clustering: DBSCAN .	68
8.8	Evaluasi Clustering	71
8.9	Internal Measures (Evaluasi Internal) .	72
8.10	Studi Kasus Evaluasi Internal	72

8.1 Reviu Unsupervised Learning

Definisi 8.1: Unsupervised Learning

Unsupervised Learning adalah paradigma pembelajaran mesin di mana algoritma belajar dari data tanpa bantuan label atau "kunci jawaban" yang sudah ada sebelumnya.

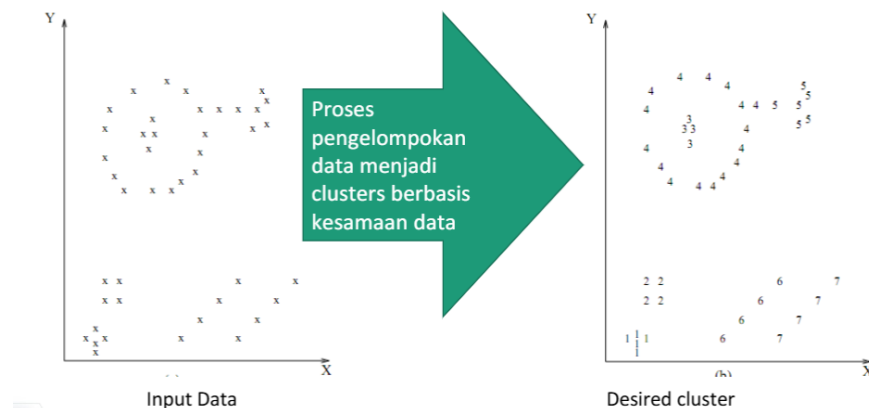
Berbeda dengan supervised learning (klasifikasi) yang memiliki label kelas seperti "Sakit" vs "Sehat", pada unsupervised learning algoritma harus "belajar sendiri" dengan mengukur seberapa mirip satu data dengan data lainnya untuk menemukan pola atau struktur tersembunyi dalam data.

8.2 Apa itu Clustering?

Definisi 8.2: Clustering

Clustering adalah proses pengelompokan sekumpulan objek data (pola) ke dalam himpunan-himpunan (disebut cluster) berdasarkan kesamaan data (data similarity).

Tujuan utama: Menemukan struktur intrinsik dalam data mentah (raw data) tanpa bantuan label yang sudah ada sebelumnya.



Gambar 8.1: Visualisasi Clustering

Konsep: Prinsip Kualitas Cluster yang Baik

Kualitas hasil clustering sangat ditentukan oleh dua metrik utama:

Definisi 8.3: High Intra-class Similarity (Compactness)

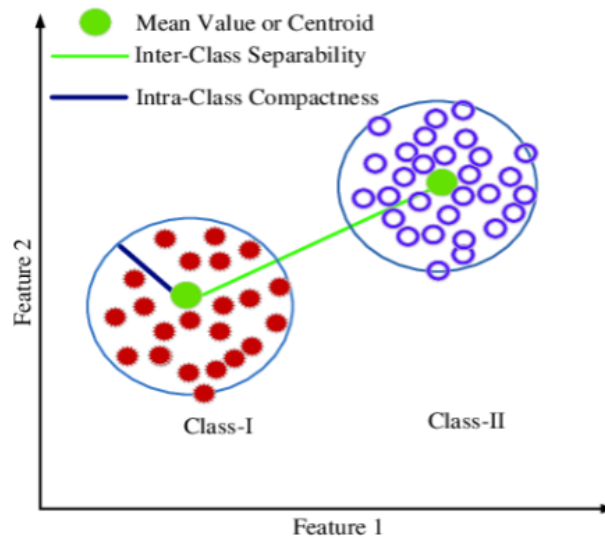
Objek-objek di dalam satu cluster yang sama haruslah semirip mungkin atau jaraknya sangat dekat satu sama lain.

Semakin rapat (compact) anggota cluster, semakin baik.

Definisi 8.4: Low Inter-class Similarity (Separation)

Objek antar cluster yang berbeda haruslah seberbeda mungkin atau jaraknya sangat jauh.

Cluster satu dengan yang lain harus terpisah secara jelas (well-separated).



Gambar 8.2: Visualisasi Natural Groups

Konsep: Mengapa Perlu Clustering? (Aplikasi Riil)

Clustering digunakan untuk eksplorasi data (data discovery) di berbagai domain:

Definisi 8.5: Aplikasi Clustering**Search Engine (Mesin Pencari):**

Masalah: Mencari kata kunci yang ambigu (misal: "Apple").

Solusi: Mengelompokkan hasil pencarian menjadi topik-topik terpisah seperti "Apple (Buah)", "Apple (Perusahaan)", "Resep", dll. (Contoh: Yippy Search).

News Aggregator:

Mengelompokkan ribuan artikel berita yang masuk setiap menit menjadi topik kejadian yang sama (misal: "Gempa Bumi Cianjur", "Pemilu 2024"). Contoh: Google News.

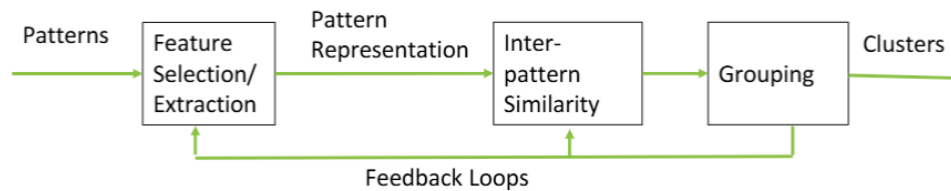
Bioinformatika (Analisis Gen):

Mengelompokkan gen yang memiliki pola ekspresi serupa dalam kondisi tertentu, yang seringkali mengindikasikan fungsi biologis yang sama atau terkait. Visualisasi sering menggunakan heatmap.

Marketing (Customer Segmentation):

Membagi basis pelanggan ke dalam segmen-segmen seperti "High Spender Young", "Low Spender Senior", dll., berdasarkan atribut Umur, Pendapatan, dan Resensi (kapan terakhir belanja).

8.3 Tahapan Proses Clustering



Gambar 8.3: Alur kerja Clustering

Konsep: Tahapan Clustering

1. Feature Selection & Extraction (Pra-pemrosesan):

- *Feature Selection*: Memilih subset fitur asli yang paling relevan untuk membedakan objek (misal: memilih fitur 'Gaji' tapi membuang 'Nama').
- *Feature Extraction*: Mentransformasi data asli menjadi fitur baru yang lebih representatif (misal: PCA).

2. Proximity Measure (Pengukuran Jarak):

Menentukan cara matematis untuk menghitung "kemiripan".

Contoh: Jarak Euclidean (garis lurus), Manhattan, atau Cosine Similarity. Pilihan jarak harus sesuai dengan domain masalah.

3. Grouping (Pengelompokan):

Menjalankan algoritma clustering.

Output bisa berupa *Hard Clustering* (setiap data pasti masuk ke 1 cluster) atau *Soft Clustering* (setiap data memiliki membership degree atau probabilitas masuk ke beberapa cluster sekaligus).

4. Data Abstraction (Opsional):

Menyederhanakan hasil cluster agar mudah dipahami manusia.

5. Assessment (Evaluasi):

Menilai apakah hasil cluster valid (bagus/buruk) menggunakan metrik evaluasi.

8.4 Representasi Cluster

Konsep: Metode Representasi Cluster

Centroid:

Cluster diwakili oleh satu titik pusat (rata-rata). Paling umum digunakan (misal: K-Means).

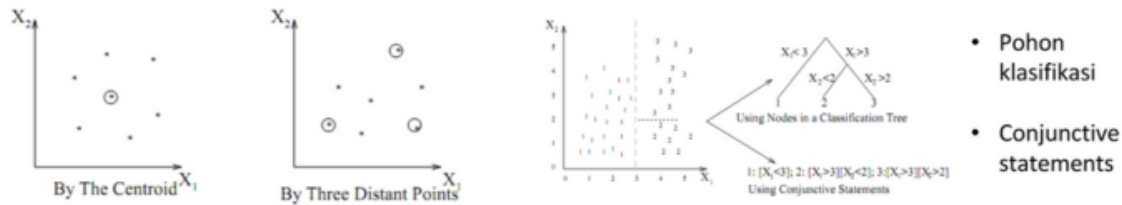
Set of Distant Points:

Cluster diwakili oleh beberapa titik terjauh (boundary) untuk menggambarkan bentuk atau batas wilayahnya.

Logical Nodes (Classification Tree):

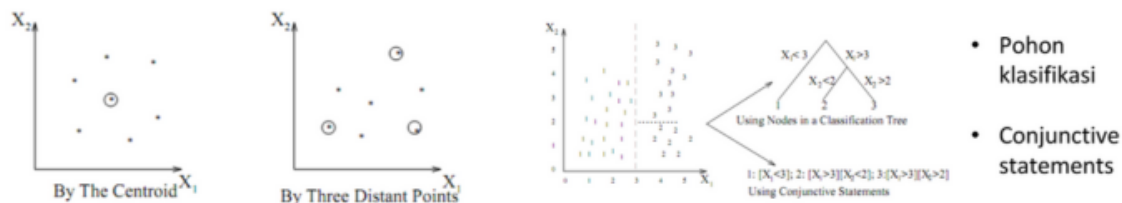
Cluster diwakili oleh aturan logika konjungtif dalam struktur pohon.

Contoh: Cluster 1 adalah data di mana ($X_1 < 3$), Cluster 2 adalah data di mana ($X_1 > 3$ AND $X_2 < 2$).



Gambar 8.4: Jenis representasi Cluster

8.5 Taksonomi Metode Clustering



Gambar 8.5: Taksonomi Metode Clustering (Fahad, 2014)

Definisi 8.6: 1. Partitioning-Based (Partisi)

Konsep: Membagi data secara langsung menjadi k kelompok yang saling lepas (exclusive).

Cara Kerja: Biasanya menggunakan teknik iteratif untuk memindahkan objek antar grup guna meminimalkan error.

Contoh: K-Means (paling populer), K-Medoids (PAM), CLARA.

Definisi 8.7: 2. Hierarchical-Based (Hirarki)

Konsep: Membuat struktur pengelompokan bertingkat seperti struktur pohon (dendrogram).

Dua Jenis Pendekatan:

- *Agglomerative (Bottom-Up)*: Mulai dengan setiap data sebagai 1 cluster, lalu digabung-gabungkan (merge) secara bertahap hingga menjadi 1 cluster besar.
- *Divisive (Top-Down)*: Mulai dengan 1 cluster besar berisi semua data, lalu dipecah-pecah (split) hingga menjadi cluster kecil-kecil.

Contoh: BIRCH, CURE, ROCK, Chameleon.

Definisi 8.8: 3. Density-Based (Berbasis Kepadatan)

Konsep: Menganggap cluster sebagai wilayah yang titik-titiknya padat, yang dipisahkan oleh wilayah yang titik-titiknya jarang (noise).

Keunggulan: Bisa menemukan cluster dengan bentuk aneh (bukan hanya bulat) dan tahan terhadap outlier.

Contoh: DBSCAN, OPTICS.

Definisi 8.9: 4. Grid-Based (Berbasis Grid)

Konsep: Membagi ruang data menjadi sel-sel grid yang terhingga (kuantisasi ruang). Operasi clustering dilakukan pada struktur grid tersebut, bukan pada titik data individu.

Keunggulan: Sangat cepat karena waktu prosesnya bergantung pada jumlah sel grid, bukan jumlah data. Cocok untuk data spasial masif.

Contoh: STING (Statistical Information Grid), WaveCluster, CLIQUE.

Definisi 8.10: 5. Model-Based (Berbasis Model Statistika)

Konsep: Mengasumsikan bahwa setiap cluster cocok dengan model distribusi statistik tertentu (biasanya Gaussian). Tujuannya adalah mencari parameter model yang paling pas dengan data.

Contoh: EM (Expectation-Maximization), SOM (Self-Organizing Maps).

8.6 Partitioning Method: K-Means Clustering

Definisi 8.11: Partitioning Method

Metode partitioning adalah pendekatan clustering yang memecah himpunan data D yang berisi n objek menjadi sejumlah k partisi (cluster) secara langsung.

Konsep: Karakteristik Utama

Eksklusif: Setiap objek data harus masuk ke dalam tepat satu cluster (kecuali pada Fuzzy Clustering). Tidak ada tumpang tindih antar kelompok.

Syarat: Setiap cluster harus berisi setidaknya satu objek.

Iterative Relocation: Algoritma ini tidak "sekali jadi". Ia bekerja dengan cara memindah-mindahkan objek dari satu cluster ke cluster lain secara berulang-ulang (iteratif) untuk memperbaiki kualitas pengelompokan hingga optimal.

Definisi 8.12: Algoritma K-Means

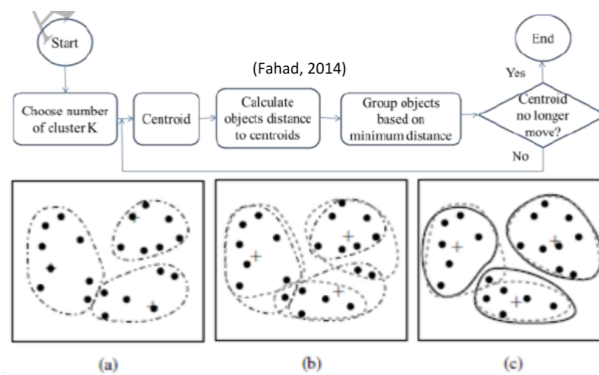
K-Means adalah algoritma *centroid-based*, artinya setiap cluster direpresentasikan oleh sebuah titik pusat yang disebut Centroid.

Logika Dasar:

Algoritma berusaha meminimalkan variasi intra-cluster. Secara matematis, ia meminimalkan fungsi objektif Sum of Squared Errors (SSE):

$$J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$$

Di mana x_i adalah titik data dan c_j adalah centroid cluster j . Semakin kecil nilai J , semakin "rapat" dan baik clusternya.



Gambar 8.6: Alur kerja Algoritma K-Means

Definisi 8.13: Langkah-langkah K-Means**1. Inisialisasi (Initialization):**

Tentukan jumlah cluster k lalu pilih k titik data secara acak (atau gunakan metode K-Means++) sebagai centroid awal. Posisi awal ini sangat krusial karena menentukan hasil akhir.

2. Penugasan (Assignment):

Ambil setiap titik data dalam dataset. Hitung jaraknya ke semua centroid yang ada (biasanya menggunakan Euclidean Distance untuk data numerik). Masukkan titik tersebut ke anggota cluster dengan centroid terdekat (jarak minimum).

3. Pembaruan (Update):

Setelah semua data ditugaskan, hitung ulang posisi centroid. Posisi centroid baru adalah rata-rata (arithmetic mean) dari semua koordinat titik data yang ada di dalam cluster tersebut. Inilah mengapa disebut "K-Means".

4. Iterasi / Berhenti:

Ulangi langkah 2 dan 3, Berhenti jika salah satu kondisi terpenuhi:

- *Konvergen*: Centroid tidak berpindah posisi lagi (atau perpindahannya sangat kecil di bawah threshold).
- *Assignment Tetap*: Tidak ada objek yang berpindah cluster.
- *Max Iteration*: Mencapai batas maksimum iterasi (misal 100 kali).

Contoh 8.6.1: Implementasi Data Kategorikal (K-Modes)

Dataset: 6 Orang (A-F) dengan 5 Atribut Biner (Ya/Tidak).

Parameter: $K = 3$.

Catatan: Contoh ini menggunakan data kategorikal/biner. Pada kasus ini, "Jarak" dihitung berdasarkan perbedaan atribut (Hamming Distance) dan "Mean" diganti dengan Mode (Nilai Mayoritas). Ini teknisnya adalah varian K-Modes.

Jawab:

0. Inisialisasi:

Kita memilih 3 data pertama sebagai Centroid Awal (Seeds):

- C_1 : A (Tidak, Ya, Ya, Tidak, Tidak)
- C_2 : B (Tidak, Tidak, Ya, Tidak, Ya)
- C_3 : C (Ya, Ya, Tidak, Ya, Tidak)

1. Iterasi Pertama (Penugasan):

Hitung jarak setiap data ke C_1, C_2, C_3 . Jarak = Jumlah atribut yang beda.

Contoh hitung jarak D ke C_1 :

D: (Tdk, Tdk, Tdk, Ya, Ya) vs C_1 : (Tdk, Ya, Ya, Tdk, Tdk).

Beda di: Atribut 2, 3, 4, 5. Total Beda = 4.

Hasil Pengelompokan Iterasi 1:

- Cluster 1: {A}
- Cluster 2: {B, D, E}
- Cluster 3: {C, F}

2. Update Centroid:

Hitung "pusat" baru dengan mencari nilai mayoritas (majority voting) di setiap kolom atribut untuk masing-masing cluster.

Centroid Baru C_1 (dari data A saja):

Tetap sama: (Tidak, Ya, Ya, Tidak, Tidak)

Centroid Baru C_2 (dari data B, D, E):

- Pengalaman: {Tdk, Tdk, Ya} → Mayoritas: Tidak
- Prog: {Tdk, Tdk, Tdk} → Mayoritas: Tidak
- B.Ing: {Ya, Tdk, Ya} → Mayoritas: Ya
- Buta Warna: {Tdk, Ya, Ya} → Mayoritas: Ya
- Menikah: {Ya, Ya, Ya} → Mayoritas: Ya

New C_2 : (Tidak, Tidak, Ya, Ya, Ya)

Centroid Baru C_3 (dari data C, F):

Lakukan hal yang sama. Jika seri (1 Ya, 1 Tidak), biasanya pilih acak atau pertahankan nilai lama.

Misal hasilnya: (Ya, Ya, Tidak, Ya, Tidak)

3. Iterasi Selanjutnya:

Gunakan New C_1 , New C_2 , New C_3 untuk menghitung ulang jarak seluruh data (A-F) dan lakukan penugasan ulang. Ulangi terus hingga anggota cluster tidak berubah.

Contoh 8.6.2: Data Numerik 2D (Standar K-Means)**Data:** A(1,1), B(2,1), C(4,3), D(5,4).**Parameter:** $K = 2$, Inisialisasi Centroid: $m_1 = (1, 1)$, $m_2 = (2, 1)$.**Jawab:****1. Iterasi 1 - Penugasan:**Hitung Jarak (Euclidean): $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

- A(1,1): Jarak ke $m_1 = 0$, ke $m_2 = 1$. Masuk Cluster 1.
- B(2,1): Jarak ke $m_1 = 1$, ke $m_2 = 0$. Masuk Cluster 2.
- C(4,3):
 - * ke m_1 : $\sqrt{(4-1)^2 + (3-1)^2} = \sqrt{9+4} = 3.6$
 - * ke m_2 : $\sqrt{(4-2)^2 + (3-1)^2} = \sqrt{4+4} = 2.8$. Masuk Cluster 2.
- D(5,4):
 - * ke m_1 : $\sqrt{(5-1)^2 + (4-1)^2} = 5$
 - * ke m_2 : $\sqrt{(5-2)^2 + (4-1)^2} = 4.2$. Masuk Cluster 2.

Hasil: Cluster 1 = {A}, Cluster 2 = {B, C, D}.**2. Update Centroid:****New m_1 :** Rata-rata A(1,1) $\rightarrow (1, 1)$.**New m_2 :** Rata-rata B(2,1), C(4,3), D(5,4).

$$x_{\text{new}} = (2 + 4 + 5)/3 = 3.67$$

$$y_{\text{new}} = (1 + 3 + 4)/3 = 2.67$$

New $m_2 = (3.67, 2.67)$.**3. Iterasi 2:**

Lanjut ke Iterasi 2 dengan centroid baru tersebut.

8.6.1 Penentuan Nilai K (The Elbow Method)**Definisi 8.14: Elbow Method**Salah satu tantangan K-Means adalah kita harus "menebak" jumlah cluster (k) di awal.**Metode Siku (Elbow Method):**

1. Jalankan K-Means dengan $k = 1$, hitung total SSE (sum of squared errors) atau WCSS.
2. Jalankan lagi dengan $k = 2$, hitung SSE.
3. Lakukan hingga $k = n$ (atau batas wajar).
4. Buat grafik garis: Sumbu X = k , Sumbu Y = SSE.
5. Pilih nilai k di mana penurunan SSE mulai melandai secara drastis (membentuk siku).

Logika: Menambah jumlah cluster pasti menurunkan error. Tapi jika penambahan cluster baru hanya mengurangi error sedikit, berarti cluster tersebut tidak signifikan (hanya memecah cluster yang sudah padat).

8.6.2 Analisis Kelemahan & Solusi

Konsep: Kelemahan K-Means dan Mitigasinya

1. Sensitif Inisialisasi

Penjelasan: Posisi awal centroid yang acak bisa menyebabkan hasil akhir terjebak di local optimum (solusi jelek).

Solusi: Gunakan K-Means++ (algoritma inisialisasi cerdas yang menyebar centroid awal) atau jalankan K-Means berulang kali dan ambil SSE terkecil.

2. Sensitif Outlier

Penjelasan: Karena menggunakan Mean (rata-rata), satu data ekstrem bisa menarik centroid menjauh dari kerumunan data asli.

Solusi: Gunakan K-Medoids (PAM) yang menggunakan titik data asli (median) sebagai pusat, bukan rata-rata hitung.

3. Bentuk Cluster

Penjelasan: K-Means mengasumsikan cluster berbentuk bola/sferis. Gagal pada cluster berbentuk bulan sabit atau cincin.

Solusi: Gunakan DBSCAN (Density-based) atau Spectral Clustering.

4. Tipe Data

Penjelasan: Hanya untuk numerik (Euclidean distance).

Solusi: Gunakan K-Modes untuk data kategorikal atau Gower Distance untuk data campuran.

8.7 Density-Based Clustering: DBSCAN

Definisi 8.15: Density-Based Clustering

Metode density-based mendekati masalah clustering dengan cara pandang yang berbeda total dari metode partisi (seperti K-Means).

Konsep: Analogi Pulau

Bayangkan data sebagai titik-titik lokasi orang di sebuah peta.

Cluster adalah wilayah perkotaan yang padat penduduk.

Noise adalah orang yang tinggal sendirian di hutan atau gurun (wilayah jarang).

Jika Anda bisa berjalan dari satu rumah ke rumah lain tanpa pernah keluar dari "zona padat", maka rumah-rumah itu berada di kota (cluster) yang sama.

Konsep: Implikasi Penting

Bentuk Fleksibel: Cluster bisa berbentuk apa saja (ular melingkar, huruf U, cincin), tidak harus bulat sferis.

Robust terhadap Outlier: Titik-titik di wilayah tepi otomatis dibuang sebagai noise, sehingga tidak merusak posisi pusat cluster (tidak seperti K-Means yang rata-ratanya bisa tertarik outlier).

Definisi 8.16: Parameter DBSCAN

Algoritma ini hanya membutuhkan dua parameter input untuk mendefinisikan apa itu "padat":

Epsilon (ϵ): Jarak Pandang

Radius lingkaran maksimum di sekitar sebuah titik untuk mencari tetangga.

Semakin besar ϵ , semakin banyak titik yang dianggap bertetangga, cluster akan membesar dan menyatu.

MinPts (Minimum Points): Ambang Kepadatan

Jumlah minimum titik (termasuk titik itu sendiri) yang harus ada dalam radius ϵ agar wilayah tersebut dianggap "Core" (padat).

Semakin besar MinPts, semakin ketat syarat kepadatan, semakin banyak data yang mungkin dianggap noise.

Definisi 8.17: Tipe-tipe Titik dalam DBSCAN**Core Point (Inti), berwarna Hijau:**

Definisi: Punya tetangga $\geq$ MinPts dalam radius ϵ .

Peran: Jantung cluster. Bisa "menularkan" keanggotaan cluster ke segala arah.

Border Point (Batas), berwarna Biru:

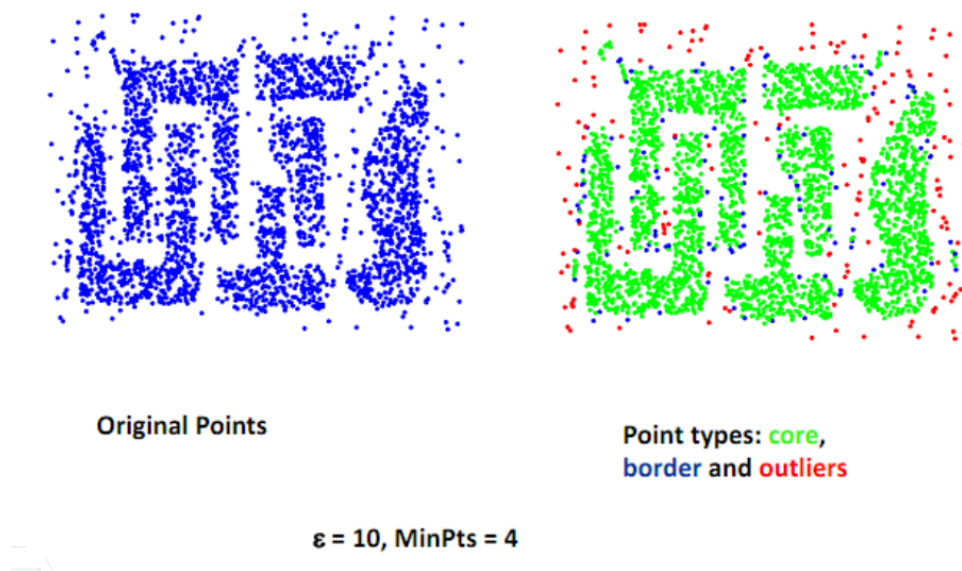
Definisi: Punya tetangga $<$ MinPts, TAPI salah satu tetangganya adalah Core Point.

Peran: Pinggiran cluster. Menjadi anggota cluster tapi tidak bisa memperluas cluster lebih jauh.

Noise / Outlier, berwarna Merah:

Definisi: Bukan Core dan bukan Border. Tidak punya tetangga Core.

Peran: Data sampah. Dibuang/diabaikan.



Gambar 8.7: Tipe-tipe titik pada DBSCAN

Contoh 8.7.1: Implementasi DBSCAN Langkah-demi-Langkah

Skenario: Terdapat 8 titik data (A1 s.d A8). Matriks di bawah ini menunjukkan Jarak Kuadrat (d^2) antar titik.

Parameter:

- $\epsilon = 2 \rightarrow$ Artinya jarak kuadrat maksimum $\epsilon^2 = 4$.
- $\text{MinPts} = 2$

Jawab:

1. Identifikasi Tetangga & Status Titik:

Cek setiap baris. Hitung berapa banyak sel yang nilainya ≤ 4 (termasuk dirinya yang bernilai 0).

- A1: Tetangga {A1}, Jumlah = 1 $\rightarrow$ **NOISE** (Kurang dari MinPts)
- A2: Tetangga {A2}, Jumlah = 1 $\rightarrow$ **NOISE**
- A3: Tetangga {A3, A5, A6}, Jumlah = 3 $\rightarrow$ **CORE** (Pusat kepadatan)
- A4: Tetangga {A4, A8}, Jumlah = 2 $\rightarrow$ **CORE**
- A5: Tetangga {A5, A3, A6}, Jumlah = 3 $\rightarrow$ **CORE**
- A6: Tetangga {A6, A3, A5}, Jumlah = 3 $\rightarrow$ **CORE**
- A7: Tetangga {A7}, Jumlah = 1 $\rightarrow$ **NOISE**
- A8: Tetangga {A8, A4}, Jumlah = 2 $\rightarrow$ **CORE**

2. Pembentukan Cluster (Ekspansi):

Mulai dari A3 (Core):

- Buat Cluster 1. Anggota awal: {A3}.
- Cek tetangga A3: {A5, A6}. Masukkan ke antrian.
- Proses A5: A5 adalah Core? Ya. Ambil tetangganya ({A3, A6}). Semua sudah di antrian. Masukkan A5 ke Cluster 1.
- Proses A6: A6 adalah Core? Ya. Ambil tetangganya ({A3, A5}). Semua sudah di antrian. Masukkan A6 ke Cluster 1.
- Antrian habis.

Hasil Cluster 1: {A3, A5, A6}.

Lanjut ke Core berikutnya yang belum dikunjungi (A4):

- Buat Cluster 2. Anggota awal: {A4}.
- Cek tetangga A4: {A8}. Masukkan ke antrian.
- Proses A8: A8 adalah Core? Ya. Ambil tetangganya ({A4}). Sudah di Cluster 2. Masukkan A8 ke Cluster 2.
- Antrian habis.

Hasil Cluster 2: {A4, A8}.

Sisa Titik (A1, A2, A7):

Status awal Noise. Cek apakah mereka tetangga dari Core?

- A1 tetangga siapa? Cuma A1. Tetap Noise.
- A2 tetangga siapa? Cuma A2. Tetap Noise.
- A7 tetangga siapa? Cuma A7. Tetap Noise.

3. Kesimpulan Akhir:

- Cluster 1: {A3, A5, A6}
- Cluster 2: {A4, A8}
- Outlier: {A1, A2, A7}

Definisi 8.18: Reachability dalam DBSCAN

Bagaimana DBSCAN tahu A terhubung ke B?

Directly Density-Reachable: A tetangga langsung B, dan B adalah Core Point. (Arah: $B \rightarrow A$).

Density-Reachable: A terhubung ke B lewat rantai Core Points. ($B \rightarrow C \rightarrow D \rightarrow A$).

Density-Connected: A dan B mungkin bukan Core, tapi keduanya bisa dijangkau dari satu Core Point O yang sama. ($A \leftarrow O \rightarrow B$). Ini mendefinisikan satu cluster utuh.

Konsep: Kelebihan DBSCAN

Bentuk Arbitrer: Bisa mendeteksi cluster bentuk cincin, spiral, ular, dll.

Otomatis Deteksi Outlier: Noise dipisahkan, tidak merusak rata-rata cluster.

Tanpa Input K: Tidak perlu menebak jumlah cluster (k) di awal.

Konsep: Kekurangan DBSCAN

Sensitif Parameter: Sangat sulit menentukan ϵ dan MinPts yang tepat tanpa pengetahuan domain.

Variasi Densitas: Gagal jika data memiliki cluster dengan kepadatan yang jauh berbeda (misal satu sangat padat, satu agak renggang).

Curse of Dimensionality: Perhitungan jarak menjadi kurang bermakna pada dimensi sangat tinggi.

8.8 Evaluasi Clustering

Konsep: Motivasi: Mengapa Evaluasi Cluster Penting?

Dalam Unsupervised Learning, kita tidak memiliki label kelas ("kunci jawaban") saat proses pelatihan. Namun, evaluasi tetap krusial untuk:

Definisi 8.19: Alasan Evaluasi Clustering

Menghindari Apophenia: Mencegah algoritma menemukan pola semu pada data acak (random noise). Algoritma clustering akan selalu membagi data, bahkan jika datanya hanya semut yang menyebar acak. Kita butuh metrik untuk tahu apakah pembagian itu bermakna.

Membandingkan Algoritma: Menentukan apakah K-Means bekerja lebih baik daripada DBSCAN untuk dataset spesifik.

Tuning Parameter: Membantu memilih parameter optimal, seperti menentukan jumlah cluster (k) terbaik pada K-Means (menggunakan Elbow Method yang berbasis evaluasi internal).

8.9 Internal Measures (Evaluasi Internal)

Konsep

Metode ini digunakan saat TIDAK ADA label kelas eksternal (ground truth). Penilaian didasarkan murni pada struktur geometri data hasil clustering.

Definisi 8.20: Dua Pilar Kualitas Internal

Cohesion (Kekompakan): Seberapa dekat/mirip objek-objek di dalam satu cluster.

Diukur dengan Within-Cluster Sum of Squares (WSS) atau SSE.

Target: Semakin kecil semakin baik (anggota cluster saling berpelukan erat).

Separation (Pemisahan): Seberapa jauh jarak antar cluster yang berbeda.

Diukur dengan Between-Cluster Sum of Squares (BSS).

Target: Semakin besar semakin baik (antar kelompok saling menjauh).

Definisi 8.21: Silhouette Coefficient (s)

Metrik yang menggabungkan cohesion dan separation untuk setiap titik data i .

Rumus:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $a(i)$: Rata-rata jarak titik i ke semua titik lain di cluster sendiri (mengukur cohesion).
- $b(i)$: Rata-rata jarak titik i ke semua titik di cluster tetangga terdekat (mengukur separation). Cluster tetangga adalah cluster lain yang rata-rata jaraknya paling minimal terhadap i .

Interpretasi Nilai (-1 s.d 1):

- *Mendekati +1*: Cluster Sangat Baik. (Jauh dari musuh, dekat dengan teman).
- *Sekitar 0*: Overlapping. Titik berada di perbatasan/bimbang antar dua cluster.
- *Negatif*: Mis-clustered. Titik lebih dekat ke cluster tetangga daripada clusternya sendiri.

8.10 Studi Kasus Evaluasi Internal

Contoh 8.10.1: Perhitungan Silhouette Coefficient

Data: Kita akan menghitung Silhouette untuk titik A1(2,5).

- Cluster 1 (Home): A1(2,5), A2(3,4), A3(4,6)
- Cluster 2 (Neighbor): B1(8,3), B2(9,2), B3(10,5)
- Cluster 3 (Other): C1(6,10), C2(7,8), C3(8,9)

Jawab:

1. **Hitung $a(A_1)$ - Cohesion:**

Rata-rata jarak A1 ke teman se-cluster (A2, A3).

$$d(A_1, A_2) = \sqrt{(2-3)^2 + (5-4)^2} = \sqrt{1+1} \approx 1.414$$

$$d(A_1, A_3) = \sqrt{(2-4)^2 + (5-6)^2} = \sqrt{4+1} \approx 2.236$$

$$a(A_1) = \frac{1.414 + 2.236}{2} = 1.825$$

2. Hitung $b(A_1)$ - Separation:

Rata-rata jarak A1 ke semua cluster lain, ambil yang paling kecil.

Ke Cluster 2:

$$d(A_1, B_1) \approx 6.32, d(A_1, B_2) \approx 7.61, d(A_1, B_3) = 8.00$$

$$\text{Rata-rata} = (6.32 + 7.61 + 8.00)/3 \approx 7.31$$

Ke Cluster 3:

$$d(A_1, C_1) \approx 6.40, d(A_1, C_2) \approx 5.83, d(A_1, C_3) \approx 7.21$$

$$\text{Rata-rata} = (6.40 + 5.83 + 7.21)/3 \approx 6.48$$

$$\text{Pilih Minimum: } b(A_1) = \min(7.31, 6.48) = 6.48$$

3. Hitung Nilai Silhouette:

$$s(A_1) = \frac{6.48 - 1.825}{\max(1.825, 6.48)} = \frac{4.655}{6.48} \approx 0.718$$

Kesimpulan: Nilai 0.718 cukup tinggi (dekat ke 1), menandakan A1 berada di cluster yang tepat dan terpisah cukup baik dari cluster tetangga.

8.10.1 External Measures (Evaluasi Eksternal)

Konsep

Metode ini digunakan ketika kita MEMILIKI label kelas asli (ground truth). Biasanya digunakan dalam riset untuk memvalidasi performa algoritma baru menggunakan dataset standar (seperti Iris dataset).

Definisi 8.22: Purity (Kemurnian)

Mengukur sejauh mana sebuah cluster hanya berisi data dari satu kategori kelas saja. Ini mirip dengan mengukur "akurasi" klasifikasi untuk setiap cluster.

Rumus:

$$\text{Purity} = \frac{1}{N} \sum_{k=1}^K \max_j (\text{jumlah anggota kelas } j \text{ di cluster } k)$$

1. Ambil kelas mayoritas di setiap cluster.
2. Jumlahkan anggota kelas mayoritas tersebut dari semua cluster.
3. Bagi dengan total data (N).

8.10.2 Studi Kasus Evaluasi Eksternal

Contoh 8.10.2: Perhitungan Purity

Total Data (N): 17 Dokumen.

Cluster: 3 buah.

Label Asli: Merah (A), Biru (B), Hijau (C).

Jawab:

1. Identifikasi Komposisi Cluster:

- Cluster I: 5 Merah, 1 Biru → Kelas Dominan: Merah (5)
- Cluster II: 1 Merah, 4 Biru, 1 Hijau → Kelas Dominan: Biru (4)
- Cluster III: 2 Merah, 3 Hijau → Kelas Dominan: Hijau (3)

2. Perhitungan Purity:

$$\text{Purity} = \frac{5 + 4 + 3}{17} = \frac{12}{17} \approx 0.706$$

3. Interpretasi:

Sekitar 70.6% data dikelompokkan dengan "benar" sesuai kelas mayoritasnya. Cluster I sangat murni (hanya 1 error), sedangkan Cluster III paling kotor (hampir seimbang antara Merah dan Hijau).

8.10.3 Perbandingan Kapan Menggunakan Apa

Konsep: Panduan Pemilihan Metode Evaluasi

Data Riil / Wild:

Metode: Internal (Silhouette, Elbow)

Alasan: Kita tidak punya label. Kita hanya bisa percaya pada geometri (kerapatan data).

Data Benchmark:

Metode: External (Purity, NMI)

Alasan: Kita punya label tersembunyi. Kita ingin menguji apakah algoritma mampu menemukan kembali label tersebut.

Menentukan K:

Metode: Internal (Elbow Method)

Alasan: Mencari keseimbangan variansi intra-cluster.